

Arbeitspaket (AP) 2: Advanced Data Preparation & Exploratory Data Analysis (EDA)

Persönliche Angaben (bitte ergänzen)

Vorname:

Nachname:

Immatrikulationsnummer:

Modul:

Data Science

Prüfungsdatum / Raum / Zeit:

11.11.2024 / Raum: SF O3.54 / 8:00 – 11:45

Erlaubte Hilfsmittel:

w.MA.XX.DS.24HS (Data Science)
Open Book, Eigener Computer, Internet-Zugang

Nicht erlaubt:

Nicht erlaubt ist der Einsatz beliebiger Formen von generativer KI (z.B. Copilot, ChatGPT) sowie beliebige Formen von Kommunikation oder Kollaboration mit anderen Menschen.

Bewertungskriterien

(max. erreichbare Punkte: 48)

Kategorie	Beschreibung	Punkteverteilung
Code nicht lauffähig oder Ergebnisse nicht sinnvoll	Der Code enthält Fehler, die verhindern, dass er ausgeführt werden kann (z.B. Syntaxfehler) oder es werden Ergebnisse ausgegeben, welche nicht zur Fragestellung passen.	0 Punkte
Code lauffähig, aber mit gravierenden Mängeln	Der Code läuft, aber die Ergebnisse sind aufgrund wesentlicher Fehler unvollständig (z.B. fehlerhaftes Erstellen von Subsets der Daten). Nur geringer Fortschritt erkennbar.	25% der max. erreichbaren Punkte
Code lauffähig, aber mit mittleren Mängeln	Der Code läuft und liefert teilweise korrekte Ergebnisse, aber es gibt grössere Fehler (z.B. Datenvisualisierungen entsprechen nicht den Vorgaben gemäss Fragestellung). Die Ergebnisse sind nachvollziehbar, aber unvollständig oder ungenau.	50% der max. erreichbaren Punkte
Code lauffähig, aber mit minimalen Mängeln	Der Code läuft und liefert ein weitgehend korrektes Ergebnis, aber kleinere Fehler (z.B. Achsenbeschriftung oder Titel in Graphik fehlt) beeinträchtigen die Vollständigkeit des Ergebnisses.	75% der max. erreichbaren Punkte

Kategorie	Beschreibung	Punkteverteilung
Code lauffähig und korrekt	Der Code läuft einwandfrei und liefert das korrekte Ergebnis ohne Mängel.	100% der max. erreichbaren Punkte

Python Libraries und Settings

```
In [1]: # Libraries
import os
import re
import random
import sqlite3
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import statsmodels.api as sm
from datetime import datetime, timedelta
from sqlalchemy import create_engine, text

# seaborn graphics settings
sns.set_theme(color_codes=True)

# Ignore warnings
import warnings
warnings.filterwarnings("ignore")

# Show current working directory
print(os.getcwd())
```

u:\Lektionen\DS_HS2024\MEP_HS2024\Block_02\Loesungen_LN_II\AP02

Vorbereitung (Dieser Teil wird nicht bewertet!)

1.) Starten Sie eine GitHub Codespaces Instanz auf Basis Ihres Forks des folgenden GitHub Repositories:

GitHub-Repository: https://github.com/mario-gellrich-zhaw/python_exploratory_data_analysis

Aufgaben (Dieser Teil wird bewertet!)

Hinweise zu den folgenden Aufgabenstellungen:

- Beachten Sie auch die zu jeder Aufgabenstellung zugehörigen Details zu Aufgabenstellung.
- Lösen Sie jede Aufgabe mit Hilfe von Python Code. Integrieren Sie den Python Code in die Code-Zellen der jeweiligen Aufgabe.
- Stellen Sie die Lösung(en) jeder Aufgabe ganz oder in Teilen (z.B. erste 5 Zeilen) eines DataFrames im Jupyter Notebook dar.
- Stellen Sie sämtliche von Ihnen erstellte Graphiken im Jupyter Notebook dar.

Aufgabe (1): Lesen Sie den Datensatz 'smartphone_data.csv' aus dem Ordner 'Data' ein, und speichern Sie diesen in einem pandas DataFrame.

Details zur Aufgabenstellung:

- Zeigen Sie die Dimension des DataFrames (Zeilen und Spalten) im Jupyter Notebook an.
- Zeigen Sie die Datentypen sämtlicher Spalten des DataFrames im Jupyter Notebook an.
- Zeigen Sie die ersten Zeilen des DataFrames im Jupyter Notebook an.
- Tipp: Berücksichtigen Sie den Pfad der Ablage der Daten im Ordner Data ('./Data/smartphone_data.csv').

(max. erreichbare Punkte: 4)

```
In [2]: # Einlesen der Daten 'smartphone_data.csv' aus dem Ordner 'Data' in ein DataFrame
df = pd.read_csv('./Data/smartphone_data.csv',
                 sep=';',
                 encoding='utf-8')

# Anzeigen der Dimension des DataFrames (Zeilen und Spalten)
print(f'Anzahl Zeilen: {df.shape[0]}; Anzahl Spalten: {df.shape[1]}', '\n')

# Anzeigen der Datentypen sämtlicher Spalten des DataFrames
print('Datentypen:\n', df.dtypes, '\n')

# Anzeigen der ersten 5 Zeilen des DataFrames
df.head()
```

Anzahl Zeilen: 5000; Anzahl Spalten: 12

Datentypen:

offer_id	int64
dealer_plz	int64
dealer_city	object
dealer_street_house_nr	object
make	object
model	object
price_raw	object
camera_resolution_raw	object
battery_life_raw	object
storage_size_raw	object
screen_size_raw	object
description_raw	object
dtype:	object

Out[2]:

	offer_id	dealer_plz	dealer_city	dealer_street_house_nr	make	model	price_raw
0	83197857	6394	Volketswil	Blaserstrasse 19	Apple	iPhone 13	CHF 2518.-
1	36687537	9532	Oberwil	Furrerstrasse 1	Vivo	Vivo V21	CHF 266.-
2	66722344	2227	Rüti	Maierstrasse 31	Vivo	Vivo V21	CHF 424.-
3	56164955	5766	Bülach	Sidlerstrasse 6	Google	Pixel 6a	CHF 597.-
4	94374605	8304	Belp	Steinerstrasse 8	Google	Pixel 6	CHF 1934.-



Hinweis!

Falls Sie den Datensatz 'smartphone_data.csv' nicht einlesen konnten, verwenden Sie die SQLite DB 'smartphone_data.db' im Order 'Data'.

Der folgende Python-Code liest die Daten aus der SQLite DB in ein pandas DataFrame mit dem Namen 'df'.

```
# Engine für Datenbankverbindung erstellen
engine = create_engine('sqlite:///./Data/smartphone_data.db')

# SQL Abfrage
df = pd.read_sql_query("""SELECT * FROM smartphone_data""",
                        con=engine)

# Schliessen der Datenbankverbindung
engine.dispose()

# Anzeigen des DataFrames (erste 5 Zeilen)
df.head()
```

Aufgabe (2): Extrahieren Sie die numerischen Werte der folgenden Variablen, und speichern Sie diese in separaten Variablen im DataFrame ab.

Details zur Aufgabenstellung:

- Extrahieren Sie die numerischen Werte der folgenden Variablen, und speichern Sie diese in separaten Variablen ab:
 - price_raw
 - camera_resolution_raw
 - battery_life_raw
 - storage_size_raw
 - screen_size_raw

- Benennen Sie die Spalten der neuen Variablen als (in Klammern ist der Datentyp angegeben):
 - price (int64)
 - camera_resolution (int64)
 - battery_life (int64)
 - storage_size (int64)
 - screen_size (float64)
- Stellen Sie ein Subset des DataFrames nur mit den neuen Spalten und den ersten 5 Zeilen im Jupyter Notebook dar.
- Tipp: Die folgenden regular expressions sind eine Hilfestellung zum Lösen der Aufgabe:

```
"CHF\s*(\d+)\. - "  
"(\d+)\s*MP"  
"(\d+)\s*hrs"  
"(\d+)\s*GB"  
"(\d+\.\d+)\s*IN"
```

(max. erreichbare Punkte: 8)

```
In [3]: # Extrahieren der numerischen Werte aus 'price_raw' und speichern in separaten V  
price = []  
for i in df['price_raw']:  
    d1 = re.findall("CHF\s*(\d+)\. -", i)  
    try:  
        d2 = int(''.join(d1))  
    except:  
        d2 = None  
    price.append(d2)  
  
# Save as new variable in the pandas data frame  
df['price'] = pd.Series(price, dtype="int64")
```

```
In [4]: # Extrahieren der numerischen Werte aus 'camera_resolution_raw' und speichern in  
camera = []  
for i in df['camera_resolution_raw']:  
    d1 = re.findall("(\d+)\s*MP", i)  
    try:  
        d2 = int(''.join(d1))  
    except:  
        d2 = None  
    camera.append(d2)  
  
# Save as new variable in the pandas data frame  
df['camera_resolution'] = pd.Series(camera, dtype="int64")
```

```
In [5]: # Extrahieren der numerischen Werte aus 'battery_life_raw' und speichern in sepa  
battery = []  
for i in df['battery_life_raw']:  
    d1 = re.findall("(\d+)\s*hrs", i)  
    try:  
        d2 = int(''.join(d1))
```

```

except:
    d2 = None
    battery.append(d2)

# Save as new variable in the pandas data frame
df['battery_life'] = pd.Series(battery, dtype="int64")

```

```

In [6]: # Extrahieren der numerischen Werte aus 'storage_size_raw' und speichern in separaten
storage = []
for i in df['storage_size_raw']:
    d1 = re.findall("(\d+)\s*GB", i)
    try:
        d2 = int(''.join(d1))
    except:
        d2 = None
    storage.append(d2)

# Save as new variable in the pandas data frame
df['storage_size'] = pd.Series(storage, dtype="int64")

```

```

In [7]: # Extrahieren der numerischen Werte aus 'screen_size_raw' und speichern in separaten
screen = []
for i in df['screen_size_raw']:
    d1 = re.findall("(\d+\.\d+)\s*IN", i)
    try:
        d2 = float(''.join(d1)) # use 'float' instead of 'int'
    except:
        d2 = None
    screen.append(d2)

# Save as new variable in the pandas data frame
df['screen_size'] = pd.Series(screen, dtype="float64")

```

```

In [8]: # Anzeigen Subset des DataFrames nur mit den neuen Variablen und den ersten 5 Zeilen
df[['price', 'camera_resolution', 'battery_life', 'storage_size', 'screen_size']]

```

```

Out[8]:

```

	price	camera_resolution	battery_life	storage_size	screen_size
0	2518	20	16	256	6.1
1	266	12	16	128	5.8
2	424	12	20	256	5.8
3	597	12	25	64	5.8
4	1934	20	23	64	5.8

Aufgabe (3): Erstellen Sie statistische Kennzahlen zu den numerischen Variablen im DataFrame.

Details zur Aufgabenstellung:

- Berechnen Sie die statistische Kennzahlen für die Variablen 'price', 'camera_resolution', 'battery_life', 'storage_size', 'screen_size'.
- Die statistische Kennzahlen, welche berechnet werden sollen sind (engl. Bezeichnungen):

- Count
 - Mean
 - Standard Deviation (std)
 - Minimum
 - 25th Percentile (= 1st Quartile)
 - 50th Percentile (= Median)
 - 75th Percentile (= 3rd Quartile)
 - Maximum
 - Skewness
- Stellen Sie die berechneten statistische Kennzahlen im Jupyter Notebook dar (es können auch verschiedene Ergebnistabellen angezeigt werden).
 - Tipp: Nicht alle Python - Methoden verwenden automatisch nur die numerischen Variablen. In diesen Fällen muss ein Subset der numerischen Variablen erstellt werden.

(max. erreichbare Punkte: 4)

```
In [9]: # Einlesen der aufbereiteten Daten (damit alle auf dem gleichen Stand sind)
engine = create_engine('sqlite:///./Data/smartphone_data_prepared.db')
df = pd.read_sql_query("""SELECT * FROM smartphone_data_prepared""", con=engine)
engine.dispose()

# Berechnung der statistischen Kennzahlen der numerischen Spalten in 'df'
print('Summary Statistic:\n', df.describe().round(2))

# Separate Berechnung der Skewness der numerischen Spalten
cols = ['price', 'camera_resolution', 'battery_life', 'storage_size', 'screen_si
skewness = df[cols].skew().round(4)
print('\nSkewness:\n', skewness)
```

Summary Statistic:

	offer_id	dealer_plz	price	camera_resolution	battery_life
count	5000.00	5000.00	5000.00	5000.00	5000.00
mean	55197271.47	5559.51	1391.03	22.99	22.56
std	25815003.01	2599.28	1359.78	21.90	5.31
min	10008043.00	1000.00	18.00	12.00	15.00
25%	33059199.00	3300.50	546.00	12.00	18.00
50%	54908941.50	5557.50	975.50	16.00	22.00
75%	77253679.00	7784.50	1763.00	20.00	26.00
max	99985448.00	9999.00	17091.00	108.00	35.00

	storage_size	screen_size
count	5000.00	5000.00
mean	211.00	6.15
std	165.66	0.36
min	64.00	5.80
25%	64.00	5.80
50%	128.00	6.10
75%	256.00	6.50
max	512.00	6.90

Skewness:

price	3.0052
camera_resolution	2.7580
battery_life	0.4858
storage_size	0.9657
screen_size	0.6616

dtype: float64

Aufgabe (4): Erstellen Sie eine Pivot-Tabelle mit den Mittelwerten der numerischen Variablen pro Hersteller.

Details zur Aufgabenstellung:

- Die Angabe zum Hersteller finden Sie in der Variable 'make'.
- Verwenden Sie die für die Mittelwertberechnung die Variablen 'price', 'camera_resolution', 'battery_life', 'storage_size', 'screen_size'.
- Stellen Sie die Pivot-Tabelle im Jupyter Notebook dar.

(max. erreichbare Punkte: 4)

```
In [10]: # Erstellen der Pivot-Tabelle
pivot_table = df.pivot_table(index='make',
                              values=['price',
                                       'camera_resolution',
                                       'battery_life',
                                       'storage_size',
                                       'screen_size'],
                              aggfunc='mean').round(2)

# Anzeigen der Pivot-Tabelle
pivot_table
```

	battery_life	camera_resolution	price	screen_size	storage_size
make					
Apple	22.61	23.23	2120.24	6.13	208.90
Google	22.48	22.32	1583.31	6.16	214.17
Samsung	22.62	22.37	1343.23	6.16	209.06
Vivo	22.65	23.80	842.75	6.15	214.25
Xiaomi	22.41	23.26	1058.12	6.13	208.87

Aufgabe (5): Erstellen Sie ein Histogramm der Variable 'price'.

Details zur Aufgabenstellung:

- Setzen Sie die Anzahl der Klassen (engl.: bins) auf den Wert 25.
- Beschriften Sie die Achsen des Histogramms, und fügen Sie einen Titel hinzu.
- Fügen Sie der Graphik ein Gitternetz (engl.: grid) hinzu.
- Zeigen Sie das Histogramm im Jupyter Notebook an.

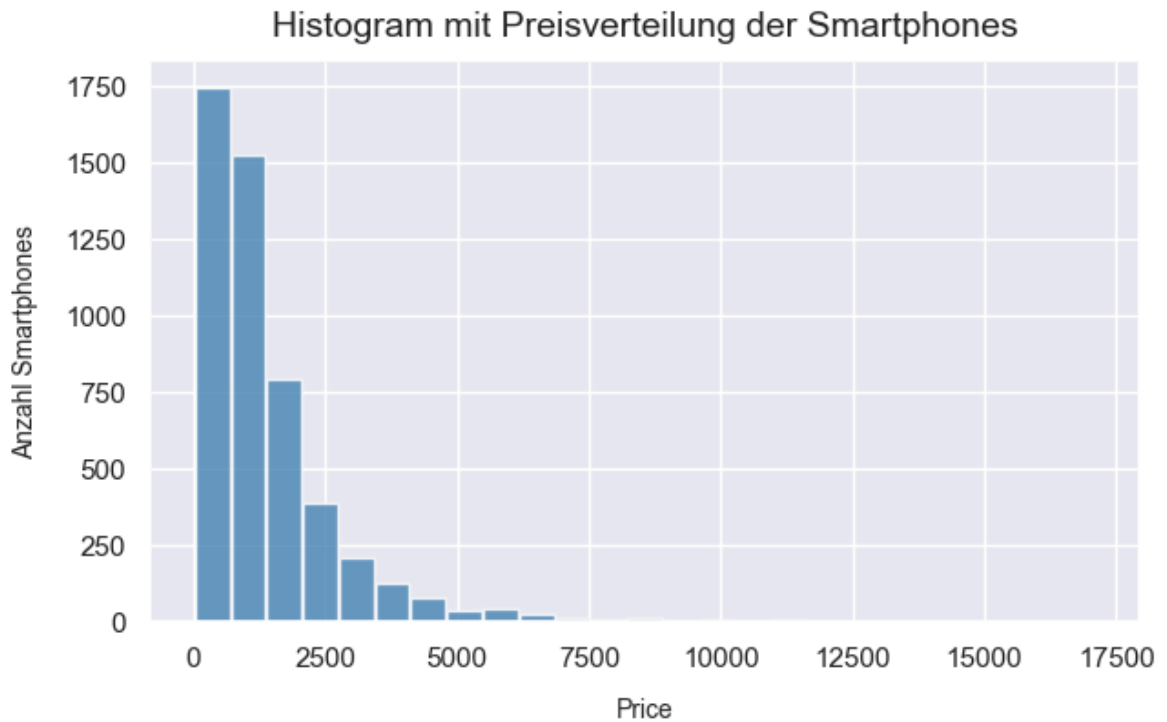
(max. erreichbare Punkte: 8)

```
In [11]: # Erstellen, Beschriften und Anzeigen des Histogramms
fig = plt.figure(figsize=(7,4))
n, bins, patches = plt.hist(x=df['price'],
                             bins=25,
                             color='steelblue',
                             alpha=0.80,
                             rwidth=0.95)

# Beschriftung der Achsen und Hinzufügen des Titels
plt.xlabel('Price', fontsize=10, labelpad=10)
plt.ylabel('Anzahl Smartphones', fontsize=10, labelpad=10)
plt.title('Histogram mit Preisverteilung der Smartphones', fontsize=14, pad=10)

# Grid
plt.grid(True)

# Zeige den Plot
plt.show()
```



Aufgabe (6): Erstellen Sie einen gruppierten Boxplot mit den Preisen pro Hersteller.

Details zur Aufgabenstellung:

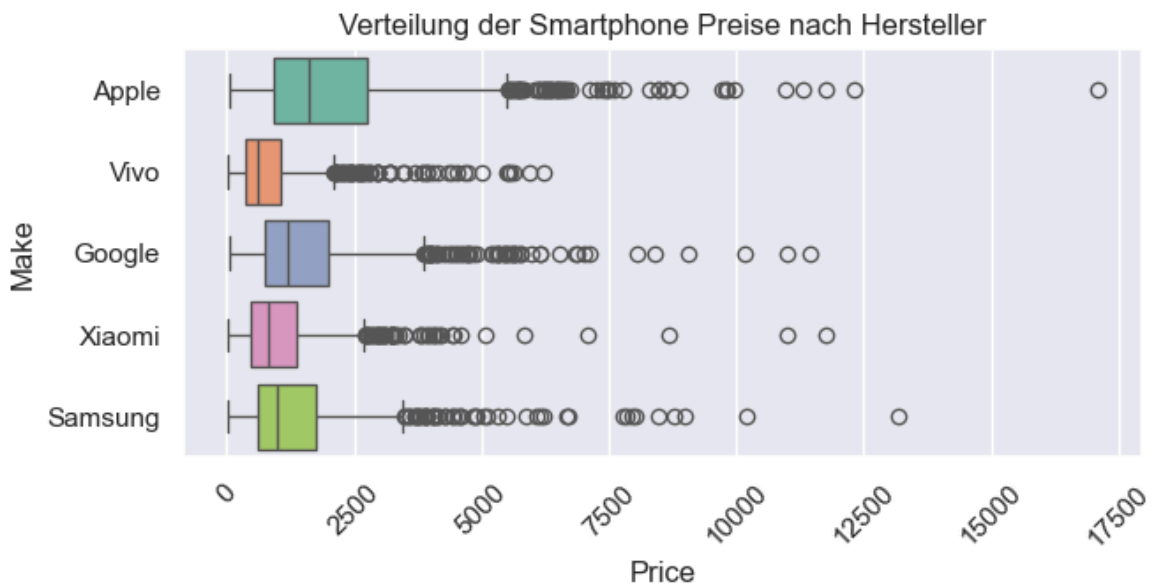
- Verwenden Sie die Python Library seaborn für die Erstellung des gruppierten Boxplots.
- Die Angabe zum Hersteller (=Gruppierungsvariable) finden Sie in der Variable 'make'.
- Beschriften Sie die Achsen und geben Sie der Graphik einen Titel.
- Stellen Sie den gruppierten Boxplot im Jupyter Notebook dar.
- Tipp: Für die Erstellung des gruppierten Boxplots können Sie Vorlagen aus der seaborn - Hilfe verwenden (vgl. Abschnitt: 'Group by a categorical variable ...'):

<https://seaborn.pydata.org/generated/seaborn.boxplot.html>

(max. erreichbare Punkte: 8)

```
In [12]: # Erstellen, Beschriften und Anzeigen eines gruppierten Boxplots der Preise pro
plt.figure(figsize=(7, 3))
sns.boxplot(x='price',
            y='make',
            data=df,
            orient='h',
            palette='Set2')
plt.title('Verteilung der Smartphone Preise nach Hersteller', fontsize=12)
plt.xlabel('Price')
plt.ylabel('Make')
```

```
plt.xticks(rotation=45)
plt.show()
```



Aufgabe (7): Erstellen Sie ein Barchart mit der Anzahl Smartphones pro Hersteller.

Details zur Aufgabenstellung:

- Ergänzen Sie den Python Code unten dort wo notwendig.
- Die Angabe zum Hersteller finden Sie in der Variable 'make'.
- Im Barchart müssen die Hersteller auf der vertikalen und die Anzahl Smartphones auf der horizontalen Achse abgebildet sein.
- Im Barchart müssen die Anzahl Smartphones pro Hersteller der Grösse nach absteigend sortiert sein.
- Beschriften Sie die Achsen des Barcharts und geben Sie dem Barchart einen Titel.

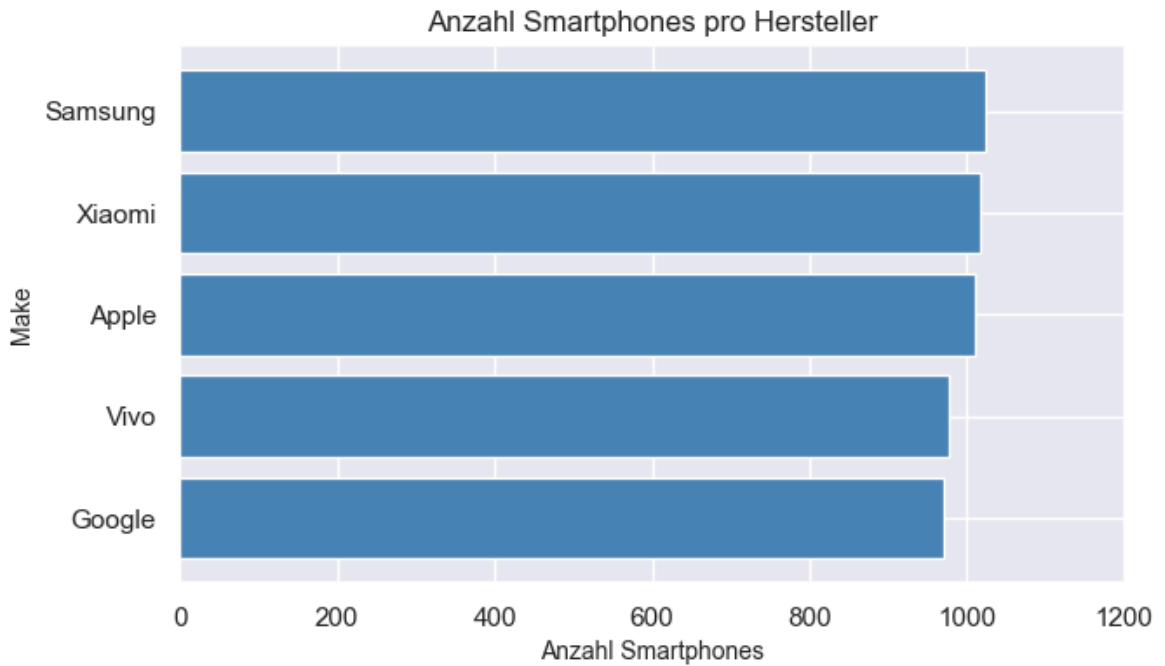
(max. erreichbare Punkte: 8)

```
In [13]: # Gruppierung der Daten nach 'make' und Berechnung der Anzahl Smartphones
df_bar = df['make'].value_counts().sort_values(ascending=True)

# Zusammenstellen der Werte für den Barchart
napart = list(df_bar.values)
index = list(df_bar.index.values)
y_pos = np.arange(len(index))

# Erstellen des Barcharts
fig, ax = plt.subplots(figsize=(7,4))
ax.barh(y_pos, napart, align='center', color='steelblue')
ax.set_yticks(y_pos, index)
ax.set_xlabel('Anzahl Smartphones', fontsize=10)
ax.set_ylabel('Make', fontsize=10)
ax.set_xlim(0, 1200)
ax.set_title('Anzahl Smartphones pro Hersteller', fontsize=12)

# Anzeigen der Graphik
plt.show()
```



Aufgabe (8): Erstellen Sie eine Correlation-Heatmap auf der Basis der numerischen Variablen.

Details zur Aufgabenstellung:

- Verwenden Sie die Variablen 'price', 'camera_resolution', 'battery_life', 'storage_size', 'screen_size'.
- Berechnen Sie auf zwei Nachkommastellen gerundete Korrelationskoeffizienten als Basis für die Correlation-Matrix und -Heatmap.
- Stellen Sie die Correlation-Matrix im Jupyter Notebook dar.
- Stellen Sie die Correlation-Heatmap im Jupyter Notebook dar.

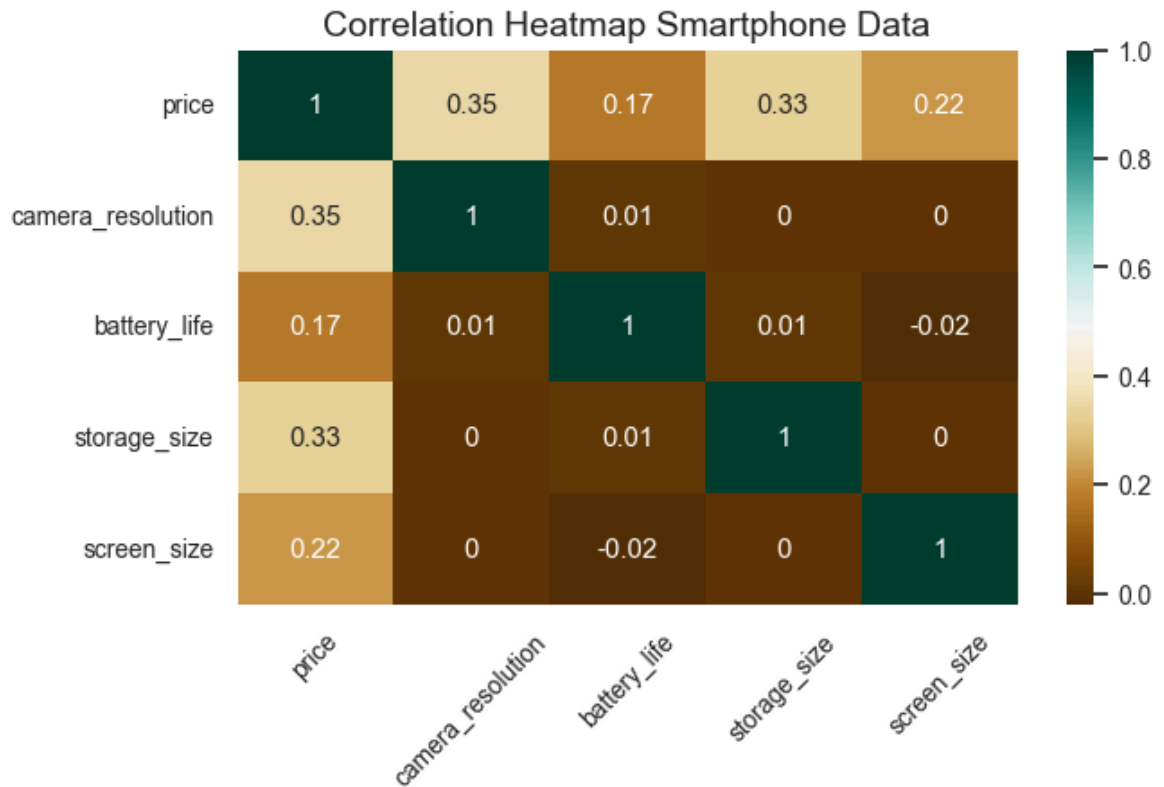
(max. erreichbare Punkte: 8)

```
In [14]: # Setzen der Schriftgrösse in seaborn
sns.set_theme(font_scale=0.9)

# Erstellen Correlation-Matrix numerischen Variablen
corr = df[['price',
            'camera_resolution',
            'battery_life',
            'storage_size',
            'screen_size']].corr().round(2)

# Erstellen der Heatmap aus Correlation-Matrix
plt.figure(figsize=(7,4))
sns.heatmap(corr,
            cmap="BrBG",
            annot=True)
plt.title('Correlation Heatmap Smartphone Data', fontsize=14)
plt.xticks(rotation=45)

# Anzeigen der Graphik
plt.show()
```



Jupyter notebook --footer info-- (please always provide this at the end of each notebook)

```
In [15]: import os
import platform
import socket
from platform import python_version
from datetime import datetime

print('-----')
print(os.name.upper())
print(platform.system(), '|', platform.release())
print('Datetime:', datetime.now().strftime("%Y-%m-%d %H:%M:%S"))
print('Python Version:', python_version())
print('IP Address:', socket.gethostbyname(socket.gethostname()))
print('-----')
```

```
-----
NT
Windows | 11
Datetime: 2024-11-17 12:37:00
Python Version: 3.12.4
IP Address: 172.27.5.229
-----
```

Exploratory data analysis (EDA) of apartment data

Libraries and settings

```
In [1]: # Libraries
import os
import random
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import statsmodels.api as sm
from datetime import datetime, timedelta

# seaborn graphics settings
sns.set_theme(color_codes=True)

# Ignore warnings
import warnings
warnings.filterwarnings("ignore")

# Show current working directory
print(os.getcwd())
```

/home/pfrei/Dokumente/GitHub/python_exploratory_data_analysis

Univariate non-graphical exploratory data analysis (EDA)

Importing the enriched apartment data

```
In [2]: # Columns to read
columns = ['web-scraper-order',
           'address_raw',
           'lat',
           'lon',
           'bfs_number',
           'bfs_name',
           'rooms',
           'area',
           'luxurious',
           'price',
           'price_per_m2',
           'pop',
           'pop_dens',
           'emp',
           'frg_pct',
           'mean_taxable_income']

# Read and select variables
df_orig = pd.read_csv("apartments_data_enriched_cleaned.csv")[columns]
```

```
# Remove duplicates
df_orig = df_orig.drop_duplicates()
df_orig.head(5)

# Remove missing values
df = df_orig.dropna()
df.head(5)
```

Out[2]:

	web-scrapers-order	address_raw	lat	lon	bfs_number	bfs_name	rooms	a
0	1693998201-1	Neuhusstrasse 6, 8630 Rüti ZH, ZH	47.252171	8.845797	118	Rüti (ZH)	3.0	4
1	1693998233-172	Widacherstrasse 5, 8630 Rüti ZH, ZH	47.252087	8.854919	118	Rüti (ZH)	3.0	11
2	1693998256-331	Widenweg 14, 8630 Rüti ZH, ZH	47.253670	8.853993	118	Rüti (ZH)	3.0	5
3	1693998265-381	Rain 1, 8630 Rüti ZH, ZH	47.259834	8.851705	118	Rüti (ZH)	4.0	11
4	1693998276-419	Bachtelstrasse 24b, 8630 Rüti ZH, ZH	47.266113	8.866872	118	Rüti (ZH)	3.0	6

Quantiles

```
In [3]: # Calculate quantiles
df[['price', 'rooms', 'area', 'price_per_m2', 'pop_dens']].quantile(
    q=[0.05, 0.10, 0.25, 0.50, 0.75, 0.90, 0.95]).round(2)
```

Out[3]:

	price	rooms	area	price_per_m2	pop_dens
0.05	1356.50	1.0	26.00	18.65	340.27
0.10	1500.00	1.5	43.00	20.36	525.66
0.25	1849.25	2.5	63.25	23.39	1044.63
0.50	2390.00	3.5	86.00	27.95	1662.60
0.75	3027.50	4.5	108.00	38.00	4778.99
0.90	3964.00	4.5	140.00	51.61	4778.99
0.95	4970.50	5.5	160.00	64.00	4778.99

Shape (number of rows and columns)

```
In [4]: # Number of rows and columns
```

```
print(df.shape)
```

(774, 16)

Data types

```
In [5]: df.dtypes
```

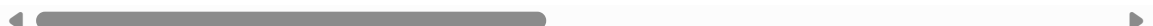
```
Out[5]: web-scraper-order      object
address_raw                  object
lat                          float64
lon                          float64
bfs_number                   int64
bfs_name                     object
rooms                       float64
area                        float64
luxurious                   int64
price                      float64
price_per_m2               float64
pop                         int64
pop_dens                   float64
emp                        float64
frg_pct                    float64
mean_taxable_income        float64
dtype: object
```

Summary statistics of numeric variables

```
In [6]: df.describe()
```

```
Out[6]:
```

	lat	lon	bfs_number	rooms	area	luxurious	
count	774.000000	774.000000	774.000000	774.000000	774.000000	774.000000	774.00
mean	47.407658	8.600456	186.255814	3.417959	88.899225	0.011628	2652.95
std	0.085577	0.117768	81.548986	1.282049	39.508987	0.107273	1281.90
min	47.195290	8.367652	1.000000	1.000000	8.000000	0.000000	86.00
25%	47.361180	8.517483	121.000000	2.500000	63.250000	0.000000	1849.25
50%	47.396210	8.563061	230.000000	3.500000	86.000000	0.000000	2390.00
75%	47.481299	8.711050	261.000000	4.500000	108.000000	0.000000	3027.50
max	47.693893	8.915933	298.000000	8.500000	282.000000	1.000000	9950.00



Statistical measures (min, max, std, mean, median, count) for selected variables

```
In [7]: # Price
print('Price:',
      'Count:', round(df.price.count(), 1),
      'Min:', round(df.price.min(), 1),
      'Max:', round(df.price.max(), 1),
      'Mean:', round(df.price.mean(), 1),
```

```

'Median:', round(df.price.median(), 1),
'Std:', round(df.price.std(), 1))

# Area
print('Area:',
      'Count:', round(df.area.count(), 1),
      'Min:', round(df.area.min(), 1),
      'Max:', round(df.area.max(), 1),
      'Mean:', round(df.area.mean(), 1),
      'Median:', round(df.area.median(), 1),
      'Std:', round(df.area.std(), 1))

```

Price: Count: 774 Min: 86.0 Max: 9950.0 Mean: 2653.0 Median: 2390.0 Std: 1281.9
 Area: Count: 774 Min: 8.0 Max: 282.0 Mean: 88.9 Median: 86.0 Std: 39.5

Skewness

```
In [8]: df[['price', 'rooms', 'area']].skew()
```

```
Out[8]: price    2.182259
rooms    0.194969
area     0.888217
dtype: float64
```

Kurtosis

```
In [9]: df[['price', 'rooms', 'area']].kurtosis()
```

```
Out[9]: price    6.933243
rooms    0.146377
area     1.963584
dtype: float64
```

Extreme values

```
In [10]: # Low costs apartments
df[df['price_per_m2'] <= 10]
```

```
Out[10]:
```

	web-scrapers-order	address_raw	lat	lon	bfs_number	bfs_name	rooms	area	luxurious	price

```
In [11]: # Very expensive apartments
df[df['price_per_m2'] >= 100]
```

```
Out[11]:
```

	web-scrapers-order	address_raw	lat	lon	bfs_number	bfs_name	rooms	area
150	1693998305-585	Steinstrasse 35, 8003 Zürich, ZH	47.366997	8.520324			261	15

Get a list of categories of categorical variable

```
In [12]: np.array(pd.Categorical(df['bfs_name']).categories)
```

```
Out[12]: array(['Adliswil', 'Aeugst am Albis', 'Affoltern am Albis', 'Altikon',  
                'Andelfingen', 'Bachenbülach', 'Bassersdorf', 'Bauma',  
                'Bonstetten', 'Brütten', 'Bülach', 'Dielsdorf', 'Dietikon',  
                'Dietlikon', 'Dättlikon', 'Dübendorf', 'Dürnten', 'Egg', 'Eglisau',  
                'Elsau', 'Embrach', 'Fehraltorf', 'Feuerthalen',  
                'Freienstein-Teufen', 'Fällanden', 'Glattfelden', 'Gossau (ZH)',  
                'Greifensee', 'Hausen am Albis', 'Hedingen', 'Herrliberg',  
                'Hettlingen', 'Hinwil', 'Hittnau', 'Hochfelden', 'Hombrechtikon',  
                'Höri', 'Hüttikon', 'Kloten', 'Knöna', 'Küsnacht (ZH)',  
                'Langnau am Albis', 'Laufen-Uhwiesen', 'Lindau', 'Lufingen',  
                'Maur', 'Meilen', 'Mettmenstetten', 'Männedorf', 'Mönchaltorf',  
                'Neerach', 'Neftenbach', 'Niederglatt', 'Niederhasli',  
                'Niederweningen', 'Nürensdorf', 'Oberengstringen', 'Oberglatt',  
                'Obfelden', 'Oetwil am See', 'Oetwil an der Limmat', 'Opfikon',  
                'Ossingen', 'Pfungen', 'Pfäffikon', 'Regensdorf', 'Rheinau',  
                'Richterswil', 'Rickenbach (ZH)', 'Rorbas', 'Russikon', 'Rümlang',  
                'Rüschlikon', 'Rüti (ZH)', 'Schlatt (ZH)', 'Schlieren',  
                'Schwerzenbach', 'Seuzach', 'Stadel', 'Stallikon', 'Steinmaur',  
                'Stäfa', 'Thalwil', 'Trüllikon', 'Uetikon', 'Urdorf', 'Uster',  
                'Volketswil', 'Wald (ZH)', 'Wallisellen', 'Wangen-Brüttisellen',  
                'Weiach', 'Weiningen (ZH)', 'Wettswil am Albis', 'Wetzikon (ZH)',  
                'Wiesendangen', 'Wila', 'Winkel', 'Winterthur', 'Zell (ZH)',  
                'Zollikon', 'Zürich'], dtype=object)
```

Multivariate non-graphical exploratory data analysis (EDA)

Cross-tabulation

```
In [13]: pd.crosstab(df['luxurious'], df['rooms'])
```

```
Out[13]:
```

	rooms	1.0	1.5	2.0	2.5	3.0	3.5	4.0	4.5	5.0	5.5	6.0	6.5	7.0	7.5	8.5
luxurious	0	50	33	38	135	44	200	28	163	3	52	2	11	2	3	1
	1	0	0	0	6	0	3	0	0	0	0	0	0	0	0	0

Pivot tables

```
In [14]: # Using pivot_table to reshape the data and calculate means  
pd.pivot_table(df[['rooms', 'price', 'price_per_m2', 'area', 'luxurious']],  
                index=['rooms', 'luxurious'],  
                values=['price', 'price_per_m2', 'area'],  
                aggfunc=(np.mean, 'count'))
```

Out[14]:

		area		price		price_per_m2	
		count	mean	count	mean	count	mean
rooms	luxurious						
1.0	0	50	32.920000	50	1306.220000	50	47.140400
1.5	0	33	35.969697	33	1963.515152	33	59.417576
2.0	0	38	55.184211	38	2120.078947	38	40.757105
2.5	0	135	66.800000	135	2382.755556	135	37.010000
	1	6	72.166667	6	3339.500000	6	45.978333
3.0	0	44	68.159091	44	1964.022727	44	28.677955
3.5	0	200	89.770000	200	2648.980000	200	29.494700
	1	3	101.666667	3	6036.666667	3	59.826667
4.0	0	28	98.464286	28	2852.357143	28	29.646429
4.5	0	163	114.926380	163	3086.570552	163	26.493436
5.0	0	3	108.000000	3	2881.666667	3	26.376667
5.5	0	52	153.250000	52	3590.653846	52	23.587500
6.0	0	2	185.000000	2	3800.000000	2	22.365000
6.5	0	11	167.363636	11	4114.272727	11	24.638182
7.0	0	2	175.000000	2	6350.000000	2	36.325000
7.5	0	3	205.000000	3	4765.000000	3	23.273333
8.5	0	1	210.000000	1	8500.000000	1	40.480000

Correlation matrix

```
In [15]: corr = df[['rooms',  
                    'area',  
                    'price',  
                    'price_per_m2',  
                    'pop_dens',  
                    'frg_pct']].corr()  
corr
```

Out[15]:

	rooms	area	price	price_per_m2	pop_dens	frg_pct
rooms	1.000000	0.837164	0.470852	-0.473667	-0.314237	-0.271542
area	0.837164	1.000000	0.612040	-0.456373	-0.262732	-0.259325
price	0.470852	0.612040	1.000000	0.287203	0.288451	0.090471
price_per_m2	-0.473667	-0.456373	0.287203	1.000000	0.619558	0.351672
pop_dens	-0.314237	-0.262732	0.288451	0.619558	1.000000	0.655017
frg_pct	-0.271542	-0.259325	0.090471	0.351672	0.655017	1.000000

Covariance matrix

In [16]:

```
cov = df[['rooms',
          'area',
          'price',
          'price_per_m2',
          'pop_dens',
          'frg_pct']].cov()

cov
```

Out[16]:

	rooms	area	price	price_per_m2	pop_dens
rooms	1.643649	42.404399	7.738317e+02	-9.039862	-7.015247e+02
area	42.404399	1560.960077	3.099798e+04	-268.411152	-1.807547e+04
price	773.831662	30997.977717	1.643292e+06	5480.634631	6.438886e+05
price_per_m2	-9.039862	-268.411152	5.480635e+03	221.599518	1.606008e+04
pop_dens	-701.524676	-18075.474981	6.438886e+05	16060.083639	3.032238e+06
frg_pct	-2.634166	-77.525117	8.775410e+02	39.611806	8.630505e+03



Univariate graphical exploratory data analysis (EDA)

Line chart (matplotlib)

In [17]:

```
# Generate time series data for rental apartment prices over the last 20 years w
x = []
y = []

# Start from 25 years ago with an initial price
start_date_trend = datetime(2000, 1, 1)
initial_price = 100

for i in range(25):
    date = start_date_trend + timedelta(days=365 * i)
    price_increase_factor = 1 + random.uniform(0.01, 0.02)
    initial_price *= price_increase_factor
```

```

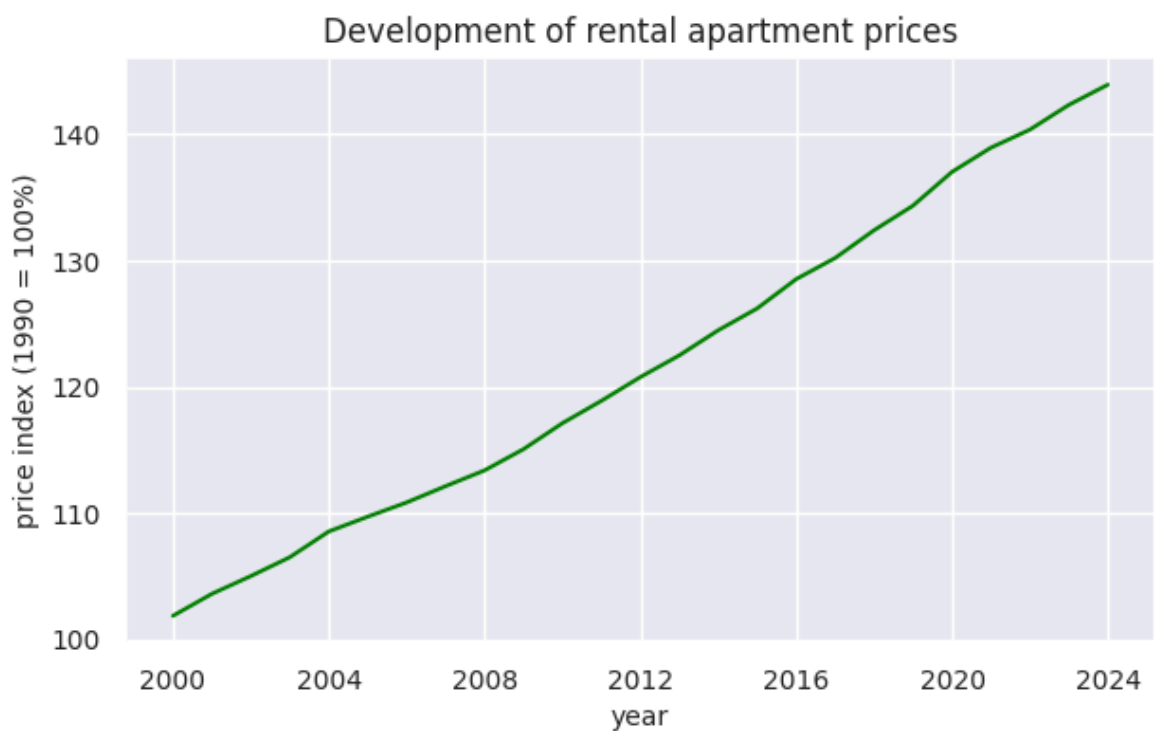
x.append(date)
y.append(round(initial_price, 2))

# Create a DataFrame to display the data
rental_prices_trend_df = pd.DataFrame({
    "Year": x,
    "Rental Price (USD)": y
})

# Create figure
fig = plt.figure(figsize=(7,4))
plt.plot(x, y, color="green")
plt.title('Development of rental apartment prices', fontsize=12)
plt.xlabel('year', fontsize=10)
plt.ylabel('price index (1990 = 100%)', fontsize=10)
plt.xticks(fontsize=10)
plt.yticks(fontsize=10)

plt.show()

```



Boxplot (seaborn)

```

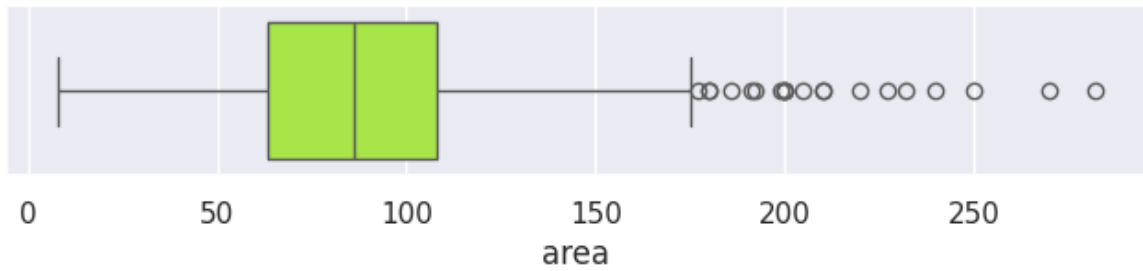
In [18]: plt.figure(figsize=(8,1.2))
plt.ticklabel_format(style='plain')
sns.boxplot(x=df['area'], color="greenyellow")

```

```

Out[18]: <Axes: xlabel='area'>

```



Histogram (matplotlib)

```
In [19]: # Plot Histogram
fig = plt.figure(figsize=(7,4))

plt.xticks(fontsize=14, rotation=0)
plt.yticks(fontsize=14, rotation=0)

n, bins, patches = plt.hist(x=df['price_per_m2'],
                             bins=25,
                             color='#42AD12',
                             alpha=0.5,
                             rwidth=0.95
                             )

plt.grid(True)
plt.ticklabel_format(style='plain')
plt.grid(axis='y', alpha=0.75)

# Set Labels
plt.xlabel('price_per_m2', fontsize=10, labelpad=10)
plt.ylabel('Number of apartments', fontsize=10, labelpad=10)
plt.title('Histogram of price_per_m2', fontsize=12, pad=10)

# Set fontsize of tick labels
plt.xticks(fontsize = 10)
plt.yticks(fontsize = 10)

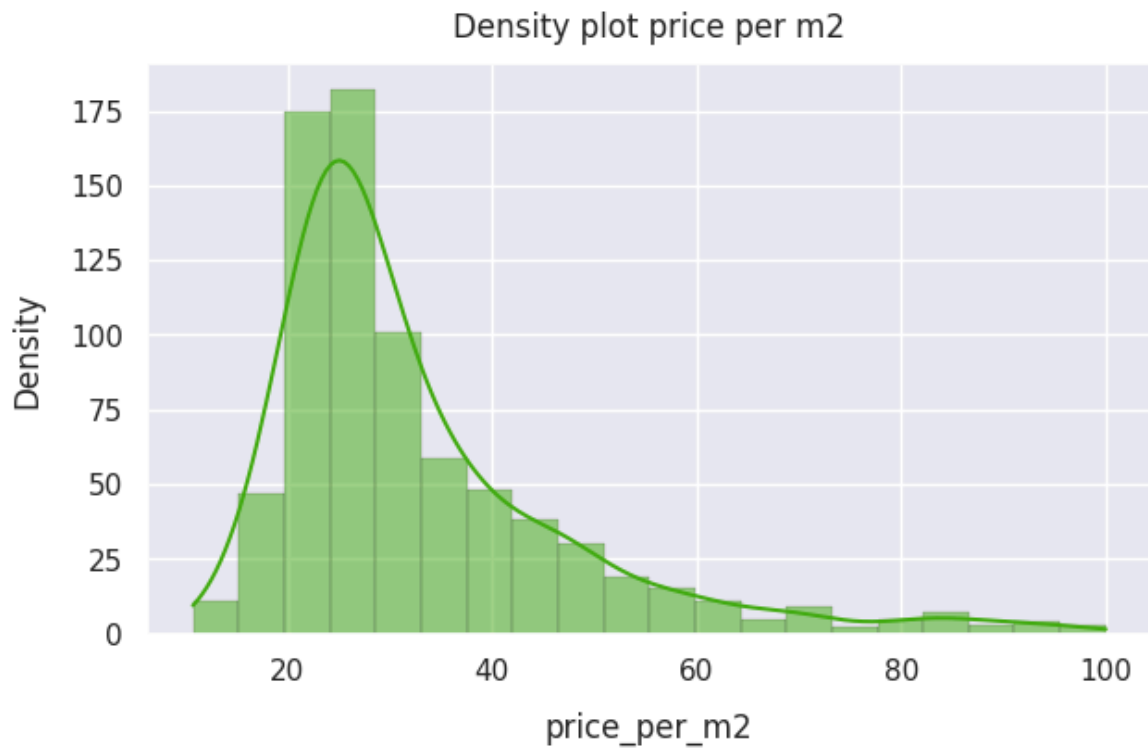
plt.show()
```



Density plot (seaborn)

```
In [20]: plt.figure(figsize=(7,4))
sns.histplot(df['price_per_m2'],
             bins=20,
             color='#42AD12',
             alpha=0.5,
             kde=True,
             edgecolor='black',
             linewidth=0.1)

plt.title('Density plot price per m2', fontsize=12, pad=10)
plt.xlabel('price_per_m2', fontsize=12, labelpad=10)
plt.ylabel('Density', fontsize=12, labelpad=10)
plt.grid(True)
plt.show()
```



Quantile-Quantile (QQ-) plot

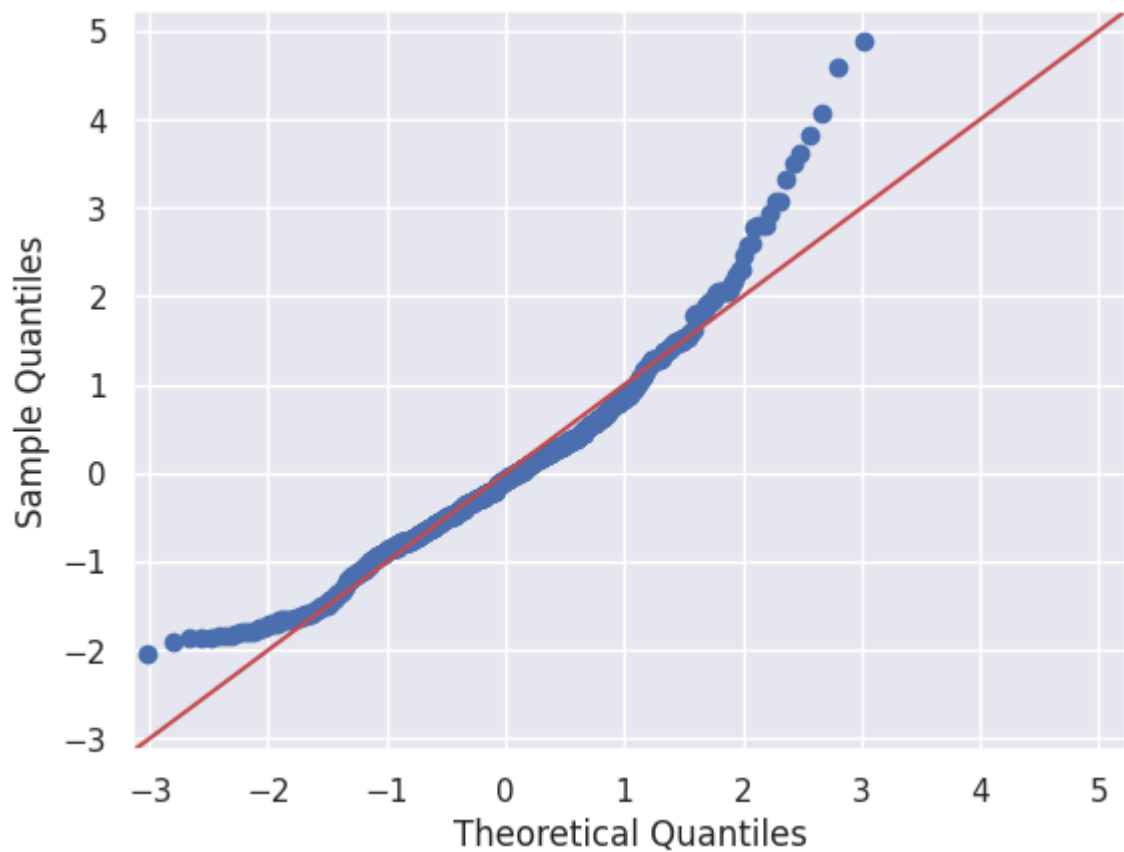
```
In [21]: # Variable (standardized)
x = 'area'
df_qq = df
df_qq['var'] = (df[x]-df[x].mean()) / df[x].std()
print(df_qq.sort_values('var')[['area', 'var']])

# Plot
sm.qqplot(df_qq['var'], line='45')
plt.show
```

	area	var
671	8.0	-2.047616
183	14.0	-1.895752
422	15.0	-1.870441
150	15.0	-1.870441
49	15.0	-1.870441
..	...	...
393	232.0	3.621980
283	240.0	3.824466
197	250.0	4.077573
675	270.0	4.583787
436	282.0	4.887515

[774 rows x 2 columns]

```
Out[21]: <function matplotlib.pyplot.show(close=None, block=None)>
```



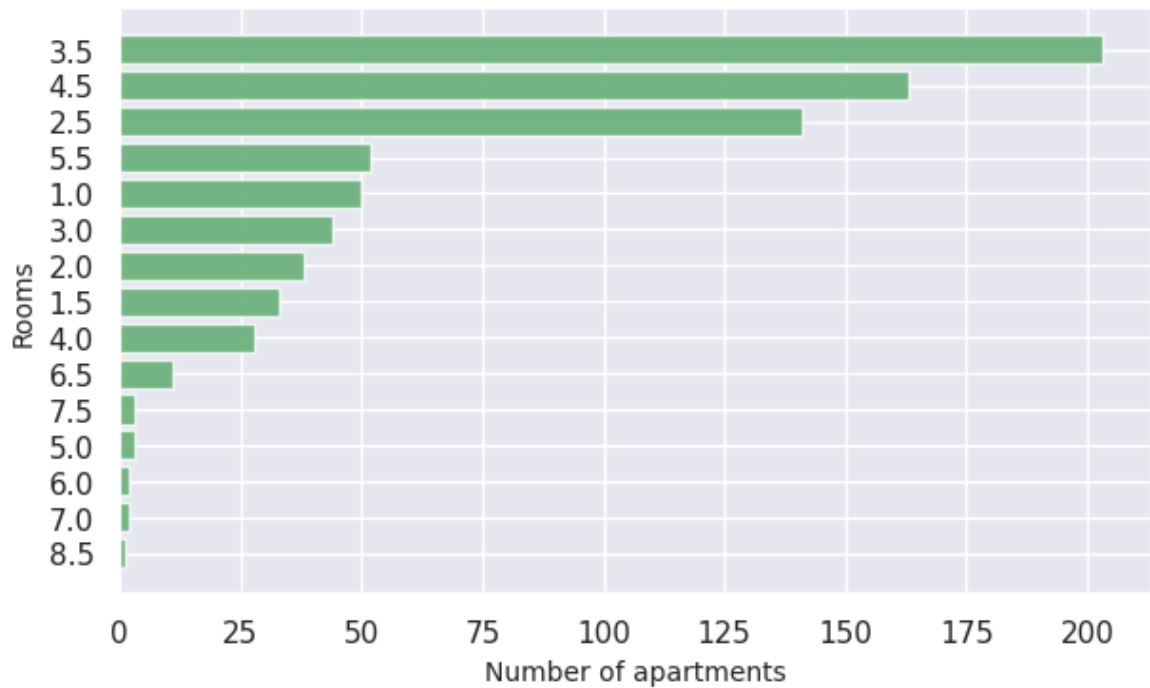
Barchart (matplotlib)

```
In [22]: # Group data by rooms (only the topmost 15 values are shown)
df_bar = df['rooms'].value_counts().nlargest(15).sort_values(ascending=True)

# Values for barchart
napart = list(df_bar.values)
index = list(df_bar.index.values)
y_pos = np.arange(len(index))

# Figure
fig, ax = plt.subplots(figsize=(7,4))
ax.barh(y_pos, napart, align='center', color='g', alpha=0.8)
ax.set_yticks(y_pos, index)
ax.set_xlabel('Number of apartments', fontsize=10)
ax.set_ylabel('Rooms', fontsize=10)

# Show graph
plt.show()
```



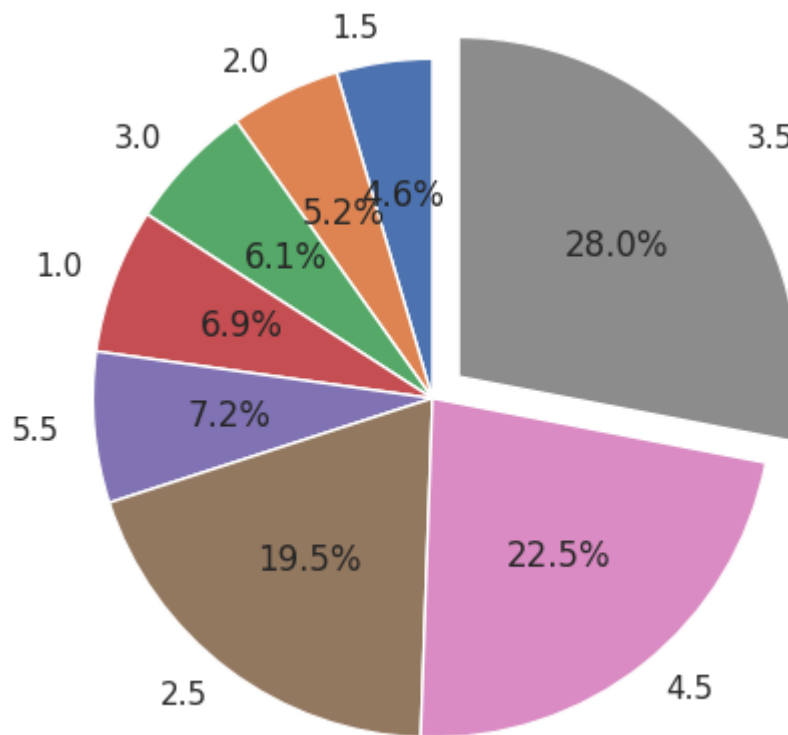
Piechart (matplotlib)

```
In [23]: # Group data by rooms (only the 8 most frequently occurencies by rooms)
df_bar = df.rooms.value_counts().nlargest(8).sort_values(ascending=True)

# Simple bar chart
sizes = list(df_bar.values)
labels = list(df_bar.index.values)
explode = (0, 0, 0, 0, 0.0, 0, 0, 0.1) # increases distance of pieces

fig1, ax1 = plt.subplots(figsize=(5,5))
ax1.pie(sizes,
        labels=labels,
        explode=explode,
        autopct='%1.1f%%',
        shadow=False,
        startangle=90)
ax1.axis('equal') # ensures that pie is drawn as a circle.

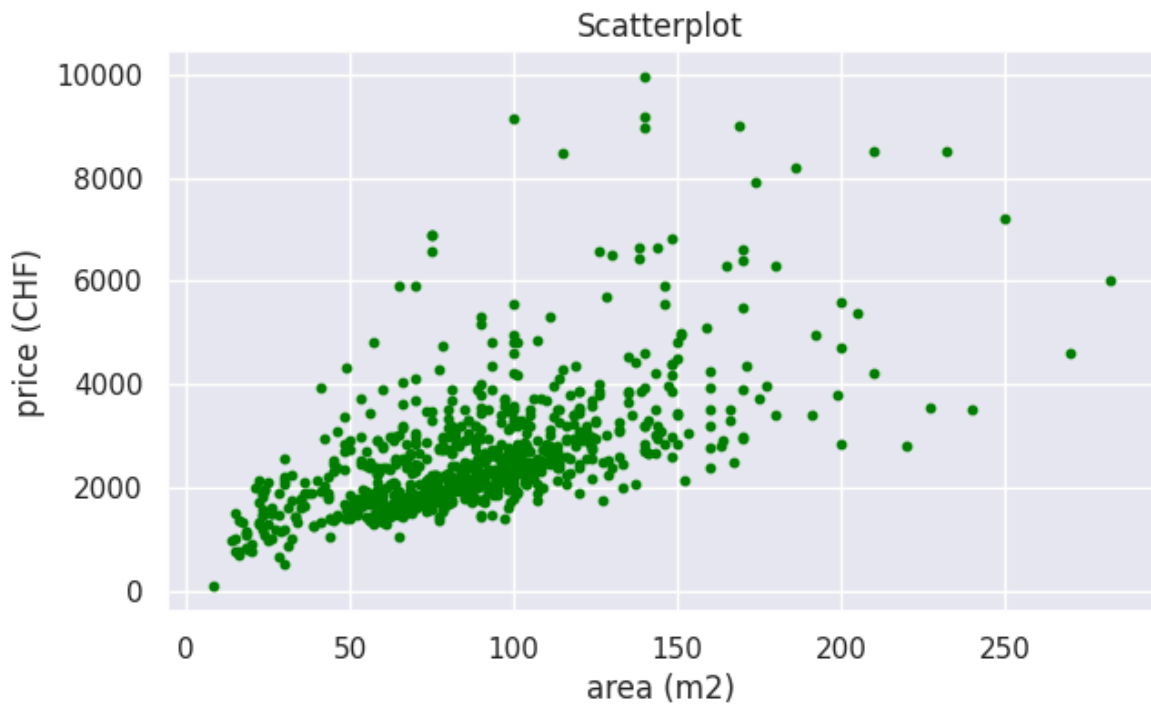
plt.show()
```



Multivariate graphical exploratory data analysis (EDA)

Scatterplot (matplotlib)

```
In [24]: plt.figure(figsize=(7,4))
plt.scatter(df['area'],
            df['price'],
            color="green",
            alpha=1.0,
            s=10)
plt.title('Scatterplot', fontsize=12)
plt.xlabel('area (m2)')
plt.ylabel('price (CHF)')
plt.show()
```



Scatterplot (matplotlib) with regression line

```
In [25]: # Subset
df_sub = df.loc[(df.price >= 1000)]
print(df_sub.shape)

# Scatterplot
plt.figure(figsize=(7,4))
plt.plot(df_sub.area,
         df_sub.price,
         'o',
         markersize=3.5,
         color="green")

# Regression line (b = slope, a=intercept)
b, a = np.polyfit(df_sub.area, df_sub.price, 1)
print(b)
print(a)

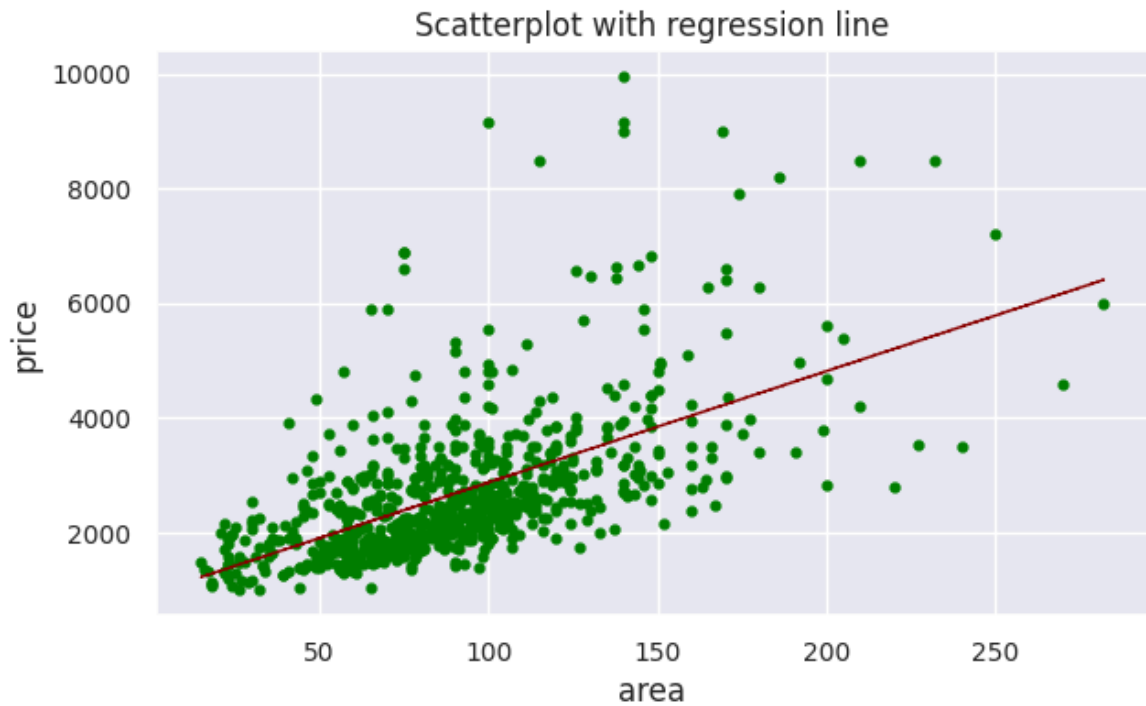
# Plot regression line
plt.plot(df_sub.area,
         b*df_sub.area + a,
         linewidth=1,
         linestyle='dashed',
         color='darkred')

# Add title and axes labels
plt.title('Scatterplot with regression line', fontsize=12)
plt.ylabel('price', fontsize=12)
plt.xlabel('area', fontsize=12)

# Set fontsize of tick labels
plt.xticks(fontsize = 10)
plt.yticks(fontsize = 10)

plt.show()
```

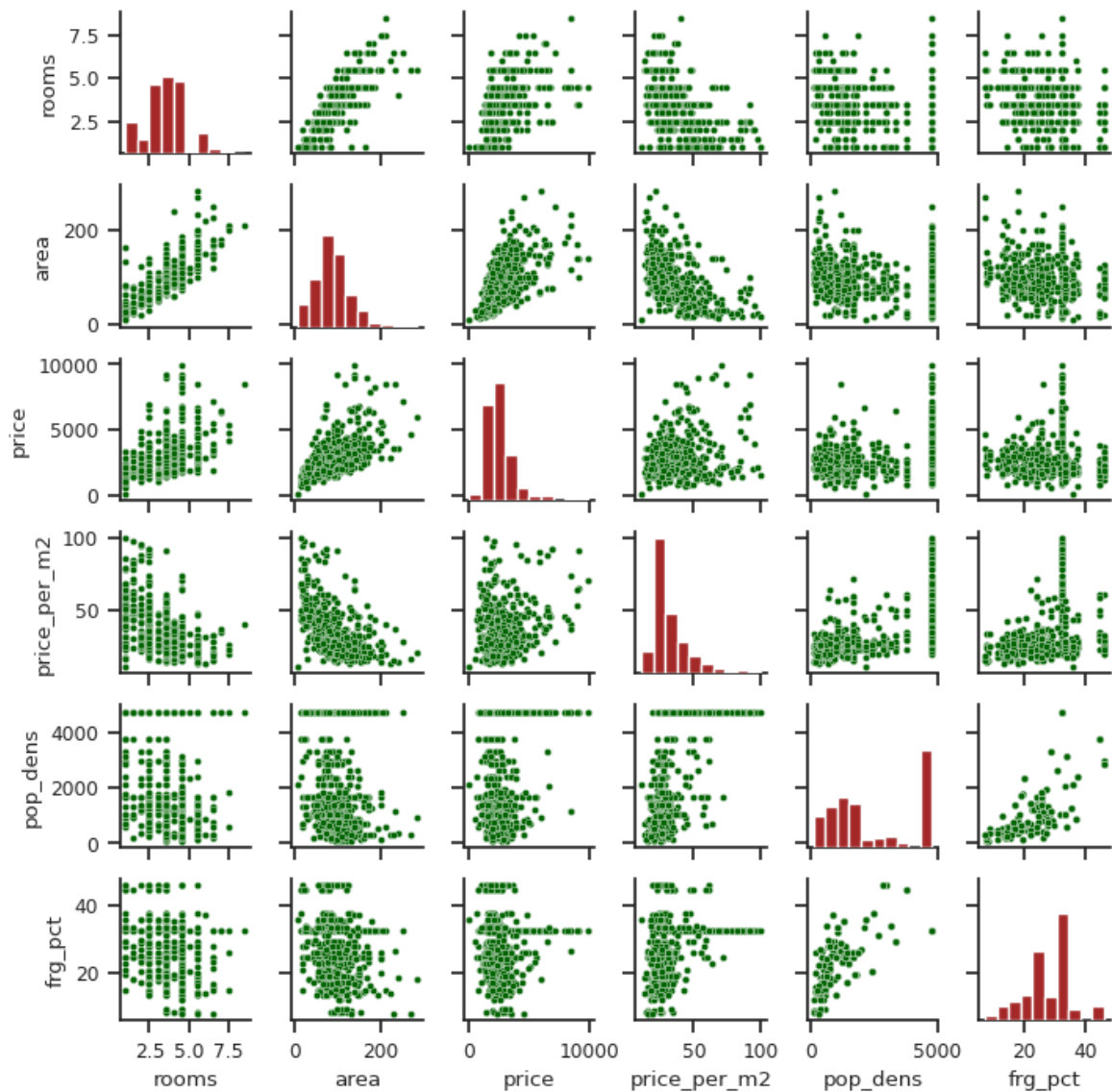
(760, 17)
19.406299563782138
938.0487789412097



Scatterplot-matrix (seaborn)

```
In [26]: sns.set_theme(style="ticks", font_scale=0.8)
g = sns.PairGrid(df[['rooms',
                    'area',
                    'price',
                    'price_per_m2',
                    'pop_dens',
                    'frg_pct']],
               height=1.2,
               aspect=1)
g.map_upper(sns.scatterplot, color='darkgreen', s=10)
g.map_lower(sns.scatterplot, color='darkgreen', s=10)
g.map_diag(plt.hist, color='brown')
```

Out[26]: <seaborn.axisgrid.PairGrid at 0x7213c87bde50>



Hexagonal binning plot (matplotlib)

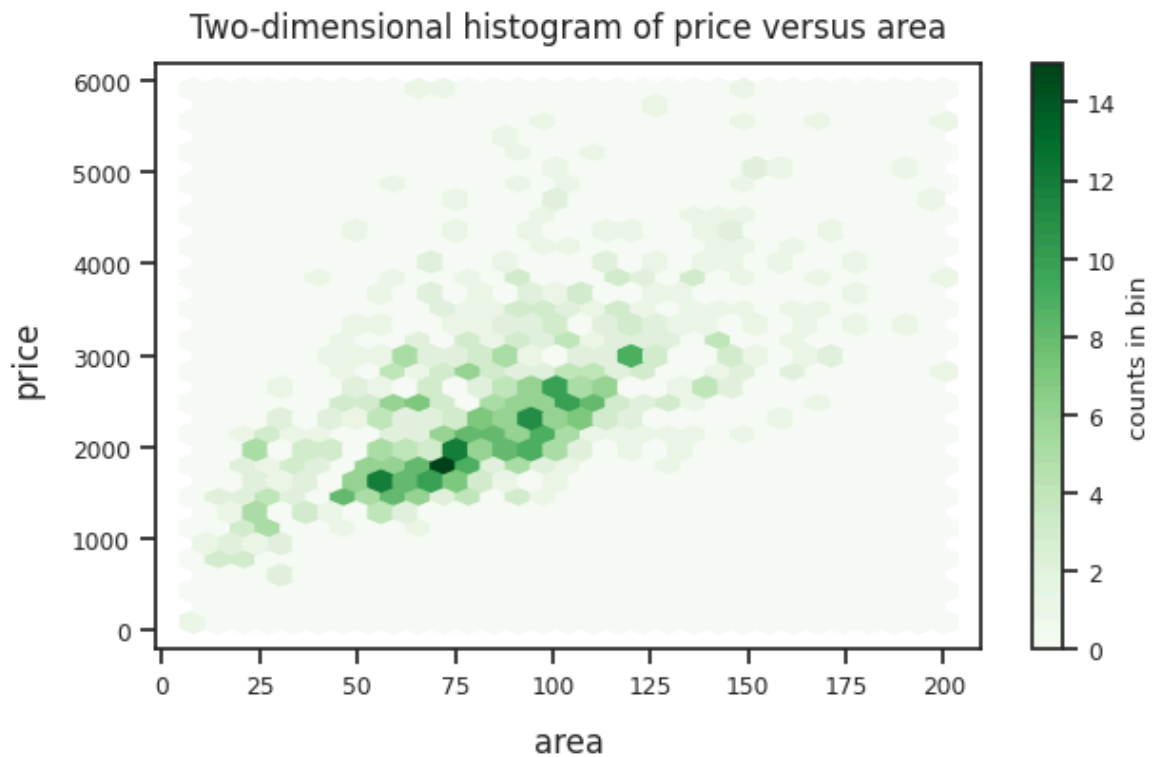
```
In [27]: # Subset
df_sub = df.loc[(df.price <= 6000) & (df.area <= 200)]
print(df_sub.shape)

# Plot
fig = plt.figure( figsize=(7,4) )
plt.hexbin(df_sub.area, df_sub.price, gridsize=30, cmap='Greens')

# Set Labels
plt.xlabel('area', fontsize=12, labelpad=10)
plt.ylabel('price', fontsize=12, labelpad=10)
plt.title('Two-dimensional histogram of price versus area', fontsize=12, pad=10)

cb = plt.colorbar(label='count in bin')
cb.set_label('counts in bin')
```

(743, 17)



Correlation heatmap (seaborn)

```
In [28]: # Set font size for plot
sns.set_theme(font_scale=0.8)

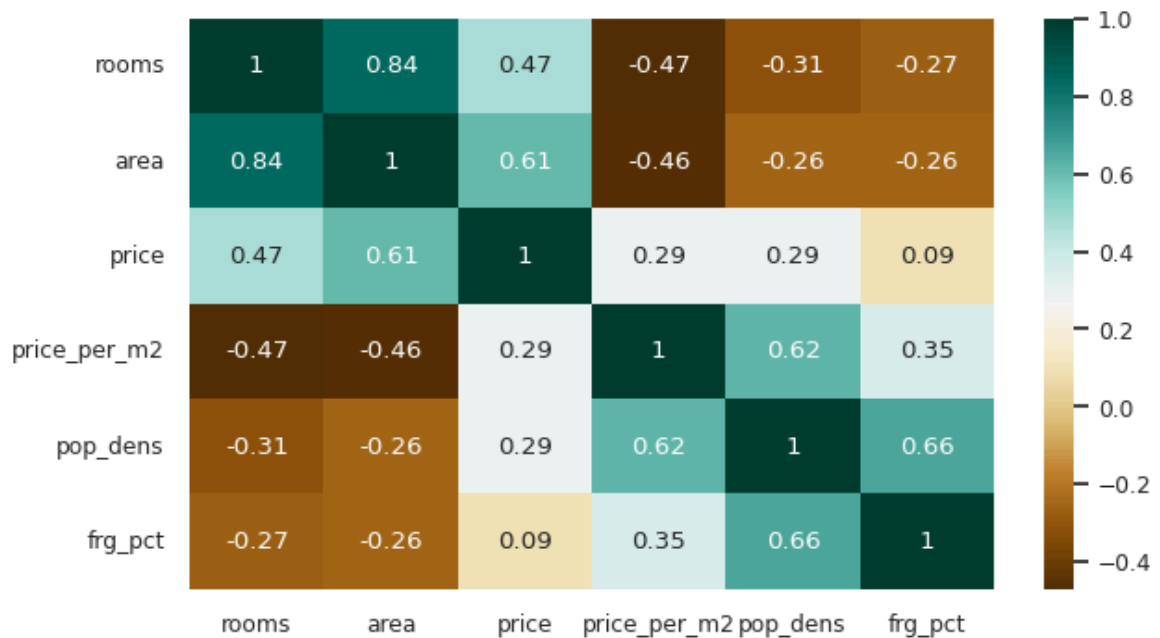
# Create correlation matrix as the basis for the heatmap
corr = df[['rooms',
           'area',
           'price',
           'price_per_m2',
           'pop_dens',
           'frg_pct']].corr().round(2)

print(corr)

# Plot heatmap
plt.figure(figsize=(7,4))
sns.heatmap(corr,
            cmap="BrBG",
            annot=True)
```

	rooms	area	price	price_per_m2	pop_dens	frg_pct
rooms	1.00	0.84	0.47	-0.47	-0.31	-0.27
area	0.84	1.00	0.61	-0.46	-0.26	-0.26
price	0.47	0.61	1.00	0.29	0.29	0.09
price_per_m2	-0.47	-0.46	0.29	1.00	0.62	0.35
pop_dens	-0.31	-0.26	0.29	0.62	1.00	0.66
frg_pct	-0.27	-0.26	0.09	0.35	0.66	1.00

Out[28]: <Axes: >



Bubble plot (seaborn)

```
In [29]: # Subset of df
df_sub = df.loc[(df['rooms'] >= 2.5) & (df['rooms'] <= 4.5)]

plt.figure(figsize=(7,4))
plt.ticklabel_format(style='plain')
cmap = sns.cubehelix_palette(dark=.3, light=3, as_cmap=True)

ax = sns.scatterplot(x="area",
                    y="price",
                    size="rooms", # determines bubble size
                    hue="pop_dens", # determines color
                    palette="Set2",
                    data=df_sub)

# Set title and axes
ax.set_title('Price vs area', fontsize = 12)
ax.set_xlabel('area', fontsize = 10)
ax.set_ylabel('price', fontsize = 10)
ax.legend([],[], frameon=False) # skip legend
```

Out[29]: <matplotlib.legend.Legend at 0x7213c02bbfe0>



Jupyter notebook --footer info-- (please always provide this at the end of each notebook)

```
In [30]: import os
import platform
import socket
from platform import python_version
from datetime import datetime

print('-----')
print(os.name.upper())
print(platform.system(), '|', platform.release())
print('Datetime:', datetime.now().strftime("%Y-%m-%d %H:%M:%S"))
print('Python Version:', python_version())
print('-----')
```

```
-----
POSIX
Linux | 6.8.0-87-generic
Datetime: 2025-11-02 20:20:09
Python Version: 3.12.12
-----
```

Exploratory data analysis (EDA) of car data

Libraries and settings

```
In [1]: # Libraries
import os
import random
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import statsmodels.api as sm
from datetime import datetime, timedelta

# seaborn graphics settings
sns.set_theme(color_codes=True)

# Ignore warnings
import warnings
warnings.filterwarnings("ignore")

# Show current working directory
print(os.getcwd())
```

/home/pfrei/Dokumente/GitHub/python_exploratory_data_analysis

Importing the car data

Importing the car data

```
In [2]: # Columns to read
columns = ["Offer_Id",
           "Type",
           "Price",
           "HP",
           "Mileage",
           "Fuel_Type",
           "Transmission",
           "Dealer_PLZ",
           "Dealer_City",
           "Dealer_Street_House_Nr",
           "Init_Regist_MY",
           "Init_Regist_Month",
           "Init_Regist_Year",
           "Init_Regist_Dt"]

# Read and select variables
df_orig = pd.read_csv("autoscout24_data_enriched_cleaned.csv", sep=",")[columns]

# Change column names to lower case
df_orig.columns = df_orig.columns.str.lower()
```

```
# Remove duplicates
df_orig = df_orig.drop_duplicates()
df_orig.head(5)

# Remove missing values
df = df_orig.dropna()
df.head(5)
```

Out[2]:

	offer_id	type	price	hp	mileage	fuel_type	transmission	dealer_plz	de
0	7324420	AUDI A5 Sportback 3.0 TDI quattro S-tronic (Li...	22500	245	75000	Diesel	Automatisiertes Schaltgetriebe	8488	Tu
1	7512768	MERCEDES-BENZ SLK 200 7G-Tronic (Cabriolet)	23749	184	46655	Benzin	Automat sequentiell	3186	D
2	7512034	MERCEDES-BENZ C 350 Avantgarde 4Matic 7G-Troni...	18500	306	138955	Benzin	Automat sequentiell	1262	
3	7512728	MERCEDES-BENZ A 45 AMG 4Matic Speedshift 7G-DC...	36000	360	43000	Benzin	Automatisiertes Schaltgetriebe	4314	Z
4	7490242	AUDI A5 Sportback 2.0 TFSI Sport quattro S-tro...	48500	252	43300	Benzin	Automatisiertes Schaltgetriebe	3250	

Univariate non-graphical exploratory data analysis (EDA)

Quantiles

```
In [3]: # Calculate Quantiles
df[['price', 'hp', 'mileage']].quantile(
    q=[0.05, 0.10, 0.25, 0.50, 0.75, 0.90, 0.95]).round(2)
```

```
Out[3]:
```

	price	hp	mileage
0.05	4000.0	90.0	13700.0
0.10	5900.0	110.0	23000.0
0.25	10900.0	150.0	45000.0
0.50	19900.0	204.0	83500.0
0.75	35900.0	306.0	130000.0
0.90	56900.0	405.0	176000.0
0.95	72500.0	469.5	206090.5

Shape (number of rows and columns)

```
In [4]: # Number of rows and columns
print(df.shape)
```

```
(3971, 14)
```

Data types

```
In [5]: # Show data types
df.dtypes
```

```
Out[5]: offer_id          int64
type              object
price             int64
hp                int64
mileage           int64
fuel_type         object
transmission      object
dealer_plz        int64
dealer_city       object
dealer_street_house_nr  object
init_regist_my    float64
init_regist_month float64
init_regist_year  float64
init_regist_dt    object
dtype: object
```

Summary statistics of numeric variables

```
In [6]: # Show summary statistics of variables
df.describe()
```

	offer_id	price	hp	mileage	dealer_plz	init_regis
count	3.971000e+03	3971.000000	3971.000000	3971.000000	3971.000000	3971.00
mean	7.242907e+06	27527.641652	235.831025	92952.412491	5522.633594	6.51
std	4.655890e+05	26670.221155	119.671388	61097.990134	2848.337824	3.38
min	1.395491e+06	160.000000	16.000000	1.000000	1000.000000	1.19
25%	7.249691e+06	10900.000000	150.000000	45000.000000	3014.000000	3.20
50%	7.408668e+06	19900.000000	204.000000	83500.000000	6003.000000	6.20
75%	7.456806e+06	35900.000000	306.000000	130000.000000	8304.500000	9.20
max	7.512843e+06	428000.000000	772.000000	370000.000000	9657.000000	12.20

Statistical measures (min, max, std, mean, median, count) for selected variables

```
In [7]: # Price
print('Price:',
      'Count:', round(df.price.count(), 1),
      'Min:', round(df.price.min(), 1),
      'Max:', round(df.price.max(), 1),
      'Mean:', round(df.price.mean(), 1),
      'Median:', round(df.price.median(), 1),
      'Std:', round(df.price.std(), 1))

# Area
print('Area:',
      'Count:', round(df.mileage.count(), 1),
      'Min:', round(df.mileage.min(), 1),
      'Max:', round(df.mileage.max(), 1),
      'Mean:', round(df.mileage.mean(), 1),
      'Median:', round(df.mileage.median(), 1),
      'Std:', round(df.mileage.std(), 1))
```

Price: Count: 3971 Min: 160 Max: 428000 Mean: 27527.6 Median: 19900.0 Std: 26670.2

Area: Count: 3971 Min: 1 Max: 370000 Mean: 92952.4 Median: 83500.0 Std: 61098.0

Skewness

```
In [8]: df[['price', 'hp', 'mileage']].skew()
```

```
Out[8]: price      4.077049
        hp         1.093046
        mileage    0.886538
        dtype: float64
```

Kurtosis

```
In [9]: df[['price', 'hp', 'mileage']].kurtosis()
```

```
Out[9]: price      36.506763
        hp         1.226153
        mileage    0.739533
        dtype: float64
```

Extreme values

```
In [10]: # Low costs cars
df[df['price'] <= 500]
```

Out[10]:

	offer_id	type	price	hp	mileage	fuel_type	transmission	dealer_plz
1651	7452246	FIAT Punto 1.4 16V Sporting (Kleinwagen)	500	95	127600	Benzin	Schaltgetriebe manuell	2000
3704	7372156	VW Golf Variant 1800 Rolling Stones (Kombi)	160	90	217650	Benzin	Schaltgetriebe manuell	1426
3905	7510439	VOLVO V70 2.5 T XC AWD Nordic (Kombi)	400	193	253000	Benzin	Automat	9553



```
In [11]: # Very expensive cars
df[df['price'] >= 250000]
```

Out[11]:

	offer_id	type	price	hp	mileage	fuel_type	transmission	dealer_
312	6978696	LAMBORGHINI Aventador S LP740-4 Roadster E- Gea...	384900	741	4000	Benzin	Automatisiertes Schaltgetriebe	80
1621	6395032	PORSCHE 911 GT2 RS (Coupé)	428000	620	8900	Benzin	Schaltgetriebe manuell	10
1824	5887777	MERCEDES- BENZ 220 SE (Cabriolet)	255000	120	1200	Benzin	Schaltgetriebe manuell	60
2714	7448261	PORSCHE 911 Speedster Manual Gearbox Chalk whi...	379000	510	14	Benzin	Schaltgetriebe	10
3876	6768872	McLAREN 675LT Coupé 3.8 V8 SSG (Coupé)	280000	675	11500	Benzin	Automatisiertes Schaltgetriebe	12

Get a list of categories of categorical variable

```
In [12]: np.array(pd.Categorical(df['fuel_type']).categories)
```

```
Out[12]: array(['Automat', 'Automatik-Getriebe', 'Benzin', 'Diesel', 'Elektro',  
                'Erdgas (CNG) / Benzin', 'Flüssiggas (LPG) / Benzin',  
                'Hybrid Benzin/Elektro', 'Hybrid Diesel/Elektro',  
                'Mild-Hybrid Benzin/Elektro', 'Mild-Hybrid Diesel/Elektro',  
                'Plug-in-Hybrid Benzin/Elektro', 'Plug-in-Hybrid Diesel/Elektro',  
                'Schaltgetriebe', 'Schaltgetriebe manuell',  
                'Voll-Hybrid Benzin/Elektro', 'Voll-Hybrid Diesel/Elektro'],  
               dtype=object)
```

Multivariate non-graphical exploratory data analysis (EDA)

Cross-tabulation

```
In [13]: pd.crosstab(df['fuel_type'], df['transmission'])
```

Out[13]:

transmission	Allrad	Automat	Automat sequentiell	Automat stufenlos	Automat stufenlos, sequentiell	Automatik- Getriebe	Autom Schal
fuel_type							
Automat	0	0	0	0	0	0	
Automatik- Getriebe	1	0	0	0	0	0	
Benzin	1	244	487	8	25	62	
Diesel	0	64	564	1	11	21	
Elektro	0	0	0	6	0	1	
Erdgas (CNG) / Benzin	0	0	0	0	0	0	
Flüssiggas (LPG) / Benzin	0	0	0	0	0	0	
Hybrid Benzin/Elektro	0	0	0	0	0	1	
Hybrid Diesel/Elektro	0	0	0	0	0	0	
Mild-Hybrid Benzin/Elektro	0	0	3	0	0	0	
Mild-Hybrid Diesel/Elektro	0	0	2	0	0	0	
Plug-in- Hybrid Benzin/Elektro	0	0	15	2	0	0	
Plug-in- Hybrid Diesel/Elektro	0	0	4	0	0	0	
Schaltgetriebe	0	0	0	0	0	0	
Schaltgetriebe manuell	0	0	0	0	0	0	
Voll-Hybrid Benzin/Elektro	0	0	0	30	1	0	
Voll-Hybrid Diesel/Elektro	0	0	1	0	0	0	

Pivot tables

```
In [14]: # Using pivot_table to reshape the data and calculate means
pd.pivot_table(df[['mileage', 'price', 'hp', 'fuel_type']],
               index=['fuel_type'],
```

```
values=['price', 'mileage', 'hp'],
aggfunc=(np.mean, 'count')).round(2)
```

Out[14]:

	hp		mileage		price	
	count	mean	count	mean	count	mean
fuel_type						
Automat	1	304.00	1	76000.00	1	49000.00
Automatik-Getriebe	1	258.00	1	37800.00	1	79999.00
Benzin	2648	247.91	2648	89429.19	2648	27889.74
Diesel	1175	197.28	1175	104776.46	1175	24976.57
Elektro	67	446.57	67	61678.73	67	51967.18
Erdgas (CNG) / Benzin	1	85.00	1	50561.00	1	5900.00
Flüssiggas (LPG) / Benzin	1	97.00	1	83000.00	1	4800.00
Hybrid Benzin/Elektro	1	339.00	1	128356.00	1	19900.00
Hybrid Diesel/Elektro	1	261.00	1	263209.00	1	34000.00
Mild-Hybrid Benzin/Elektro	3	269.67	3	19536.67	3	60130.00
Mild-Hybrid Diesel/Elektro	2	340.00	2	18295.00	2	82250.00
Plug-in-Hybrid Benzin/Elektro	26	278.69	26	44840.38	26	40159.62
Plug-in-Hybrid Diesel/Elektro	4	286.00	4	68871.75	4	28622.50
Schaltgetriebe	3	242.67	3	107500.33	3	28383.33
Schaltgetriebe manuell	1	125.00	1	65000.00	1	31900.00
Voll-Hybrid Benzin/Elektro	31	168.48	31	67621.29	31	23470.65
Voll-Hybrid Diesel/Elektro	5	200.20	5	80300.00	5	17837.40

Correlation matrix

In [15]:

```
corr = df[['mileage',
           'price',
           'hp'],
          ].corr()
corr
```

Out[15]:

	mileage	price	hp
mileage	1.000000	-0.436815	-0.126362
price	-0.436815	1.000000	0.614575
hp	-0.126362	0.614575	1.000000

Covariance matrix

```
In [16]: cov = df[['mileage',
                  'price',
                  'hp'],
                  ].cov()
cov
```

```
Out[16]:
```

	mileage	price	hp
mileage	3.732964e+09	-7.117892e+08	-9.239156e+05
price	-7.117892e+08	7.113007e+08	1.961515e+06
hp	-9.239156e+05	1.961515e+06	1.432124e+04

Univariate graphical exploratory data analysis (EDA)

Line chart (matplotlib)

```
In [17]: # Generate time series data for mean car prices over the last 20 years
x = []
y = []

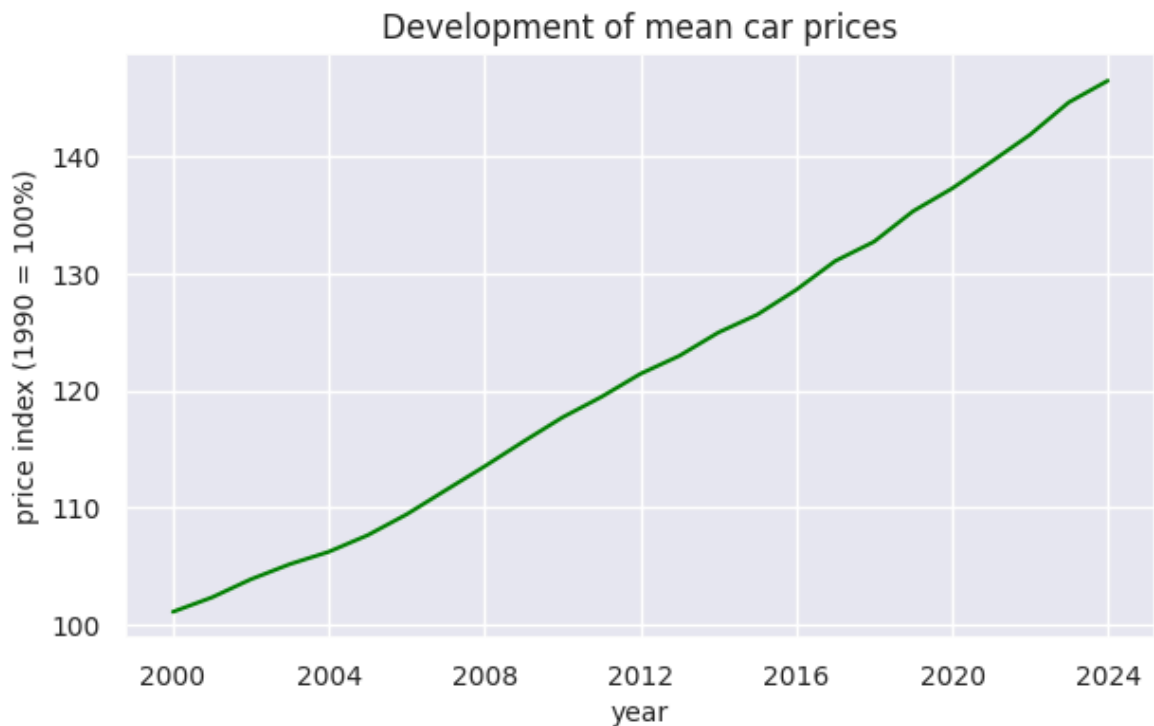
# Start from 25 years ago with an initial price
start_date_trend = datetime(2000, 1, 1)
initial_price = 100

for i in range(25):
    date = start_date_trend + timedelta(days=365 * i)
    price_increase_factor = 1 + random.uniform(0.01, 0.02)
    initial_price *= price_increase_factor
    x.append(date)
    y.append(round(initial_price, 2))

# Create a DataFrame to display the data
rental_prices_trend_df = pd.DataFrame({
    "Year": x,
    "Price (USD)": y
})

# Create figure
fig = plt.figure(figsize=(7,4))
plt.plot(x, y, color="green")
plt.title('Development of mean car prices', fontsize=12)
plt.xlabel('year', fontsize=10)
plt.ylabel('price index (1990 = 100%)', fontsize=10)
plt.xticks(fontsize=10)
plt.yticks(fontsize=10)

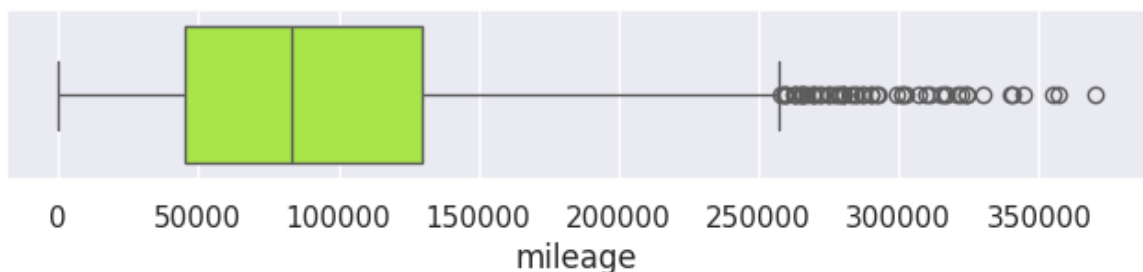
plt.show()
```



Boxplot (seaborn)

```
In [18]: plt.figure(figsize=(8,1.2))
plt.ticklabel_format(style='plain')
sns.boxplot(x=df['mileage'], color="greenyellow")
```

```
Out[18]: <Axes: xlabel='mileage'>
```



Histogram (matplotlib)

```
In [19]: # Plot Histogram
fig = plt.figure( figsize=(7,4))

plt.xticks(fontsize=14, rotation=0)
plt.yticks(fontsize=14, rotation=0)

n, bins, patches = plt.hist(x=df['price'],
                             bins=25,
                             color='#42AD12',
                             alpha=0.5,
                             rwidth=0.95
                             )

plt.grid(True)
plt.ticklabel_format(style='plain')
plt.grid(axis='y', alpha=0.75)
```

```
# Set Labels
plt.xlabel('price', fontsize=10, labelpad=10)
plt.ylabel('Number of cars', fontsize=10, labelpad=10)
plt.title('Histogram of price', fontsize=12, pad=10)

# Set fontsize of tick labels
plt.xticks(fontsize = 10)
plt.yticks(fontsize = 10)

plt.show()
```



Density plot (seaborn)

```
In [20]: plt.figure(figsize=(7,4))
sns.histplot(df['price'],
             bins=20,
             color='#42AD12',
             alpha=0.5,
             kde=True,
             edgecolor='black',
             linewidth=0.1)

plt.title('Density plot of price', fontsize=12, pad=10)
plt.xlabel('price', fontsize=12, labelpad=10)
plt.ylabel('Density', fontsize=12, labelpad=10)
plt.grid(True)
plt.show()
```



Quantile-Quantile (QQ-) plot

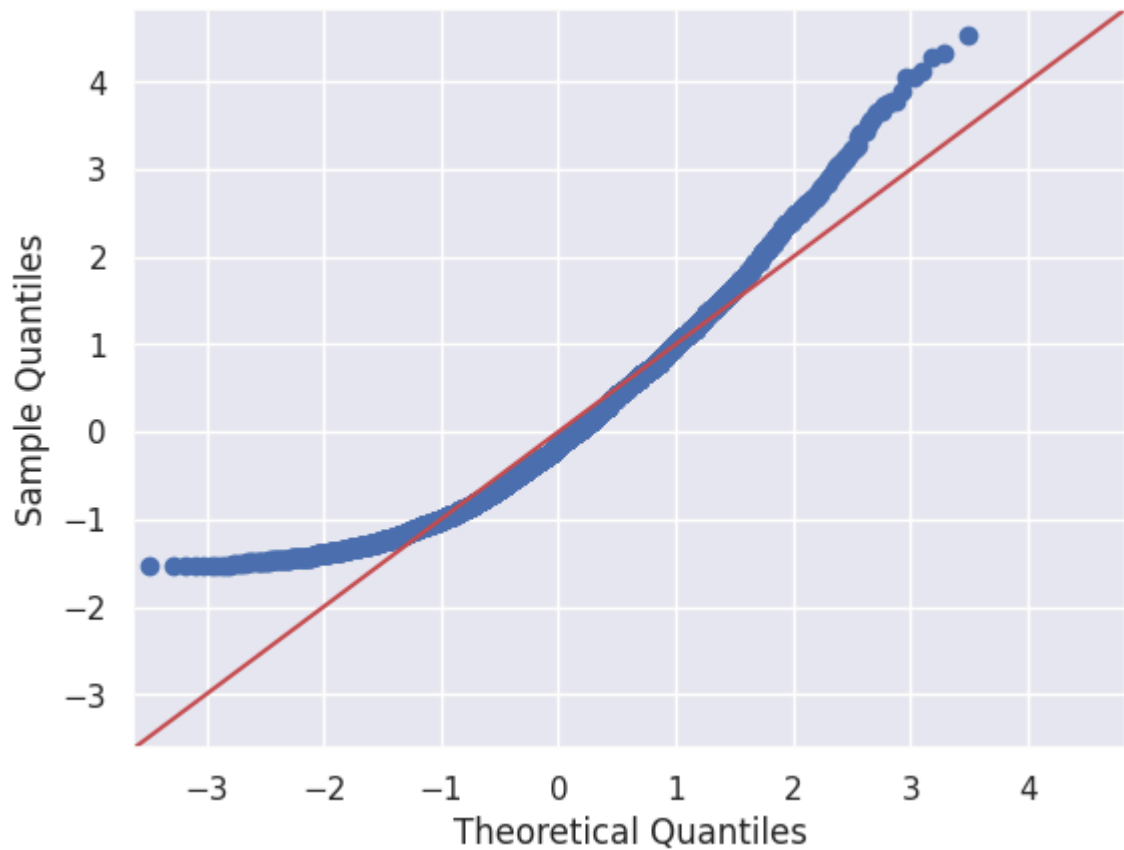
```
In [21]: # Variable (standardized)
x = 'mileage'
df_qq = df
df_qq['var'] = (df[x]-df[x].mean()) / df[x].std()
print(df_qq.sort_values('var')[['mileage', 'var']])

# Plot
sm.qqplot(df_qq['var'], line = '45')
plt.show
```

	mileage	var
1140	1	-1.521350
832	1	-1.521350
3418	10	-1.521202
3618	10	-1.521202
2695	11	-1.521186
...	...	...
2597	340890	4.058032
3073	344500	4.117117
2723	355000	4.288972
1288	356673	4.316355
988	370000	4.534480

[3971 rows x 2 columns]

```
Out[21]: <function matplotlib.pyplot.show(close=None, block=None)>
```



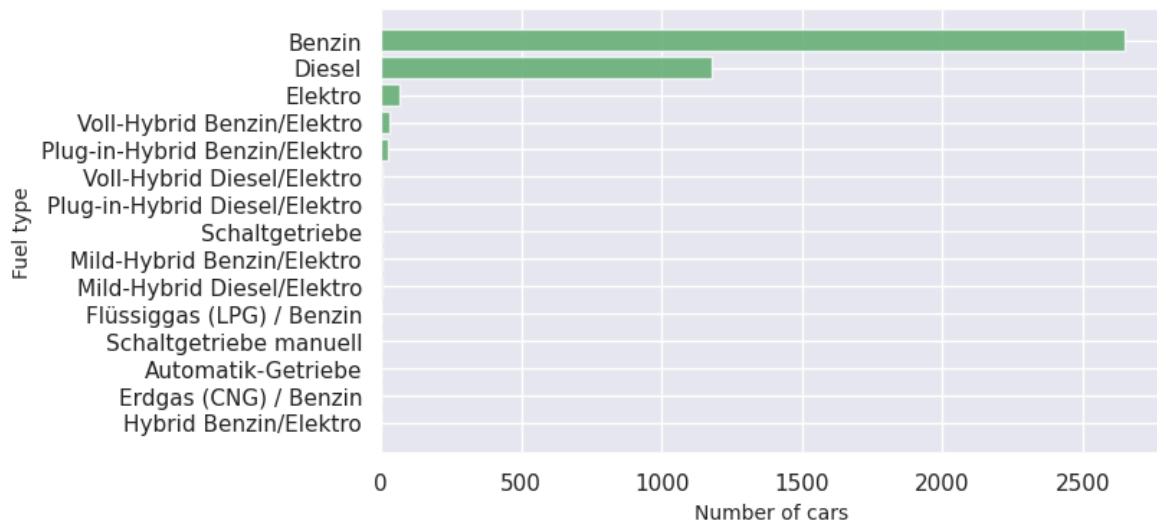
Barchart (matplotlib)

```
In [22]: # Group data by fuel_type (only the topmost 15 values are shown)
df_bar = df['fuel_type'].value_counts().nlargest(15).sort_values(ascending=True)

# Values for barchart
napart = list(df_bar.values)
index = list(df_bar.index.values)
y_pos = np.arange(len(index))

# Figure
fig, ax = plt.subplots(figsize=(7,4))
ax.barh(y_pos, napart, align='center', color='g', alpha=0.8)
ax.set_yticks(y_pos, index)
ax.set_xlabel('Number of cars', fontsize=10)
ax.set_ylabel('Fuel type', fontsize=10)

# Show graph
plt.show()
```



Piechart (matplotlib)

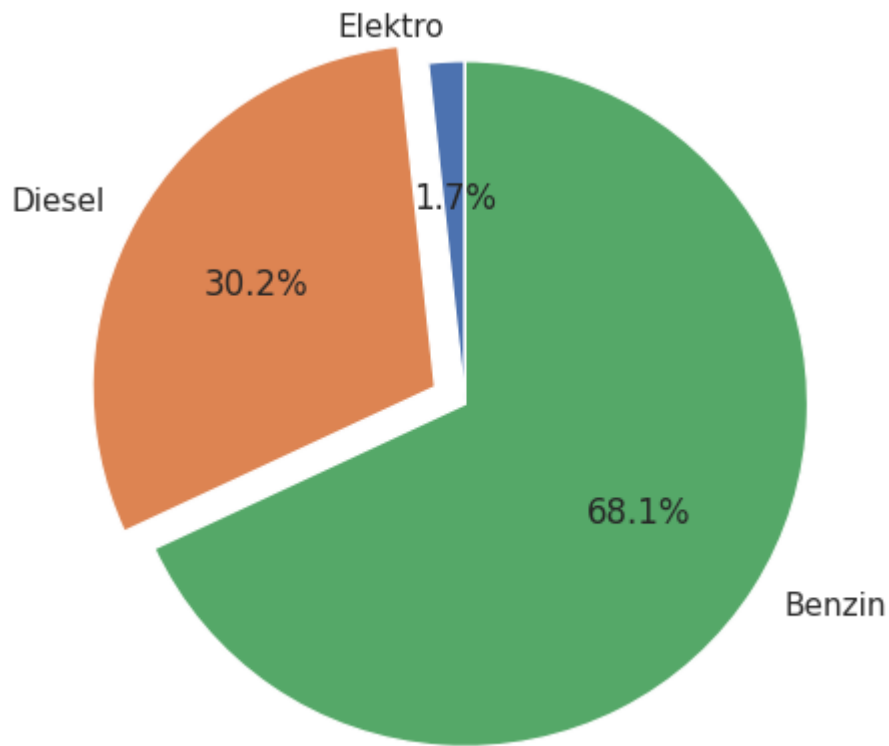
```
In [23]: # Subset of cars with fuel type
df_fuel = df[df['fuel_type'].isin(['Diesel', 'Benzin', 'Elektro'])]

# Group data (only the 8 most frequently occurrences by fuel type)
df_bar = df_fuel.fuel_type.value_counts().nlargest(8).sort_values(ascending=True)

# Simple bar chart
sizes = list(df_bar.values)
labels = list(df_bar.index.values)
explode = (0, 0.1, 0) # increases distance of pieces

fig1, ax1 = plt.subplots(figsize=(5,5))
ax1.pie(sizes,
        labels=labels,
        explode=explode,
        autopct='%1.1f%%',
        shadow=False,
        startangle=90)
ax1.axis('equal') # ensures that pie is drawn as a circle.

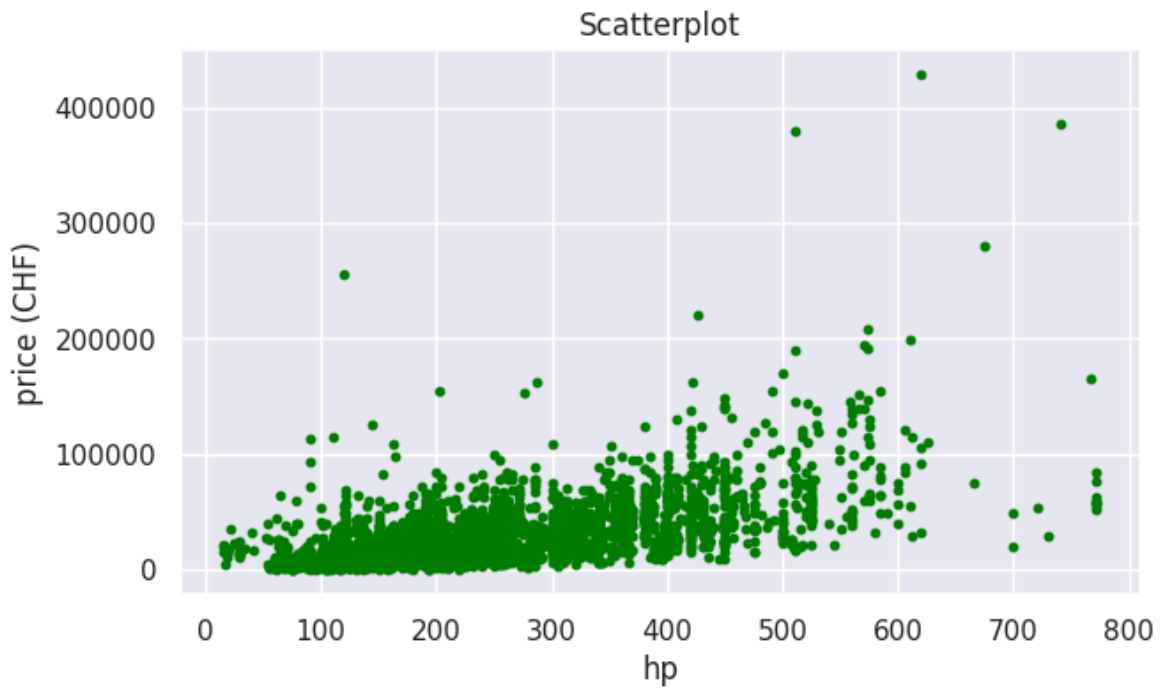
plt.show()
```



Multivariate graphical exploratory data analysis (EDA)

Scatterplot (matplotlib)

```
In [24]: plt.figure(figsize=(7,4))
plt.scatter(df['hp'],
            df['price'],
            color="green",
            alpha=1.0,
            s=10)
plt.title('Scatterplot', fontsize=12)
plt.xlabel('hp')
plt.ylabel('price (CHF)')
plt.show()
```



Scatterplot (matplotlib) with regression line

```
In [25]: # Subset
df_sub = df.loc[(df.price >= 1000)]
print(df_sub.shape)

# Scatterplot
plt.figure(figsize=(7,4))
plt.plot(df_sub.hp,
         df_sub.price,
         'o',
         markersize=3.5,
         color="green")

# Regression line (b = slope, a=intercept)
b, a = np.polyfit(df_sub.hp, df_sub.price, 1)
print(b)
print(a)

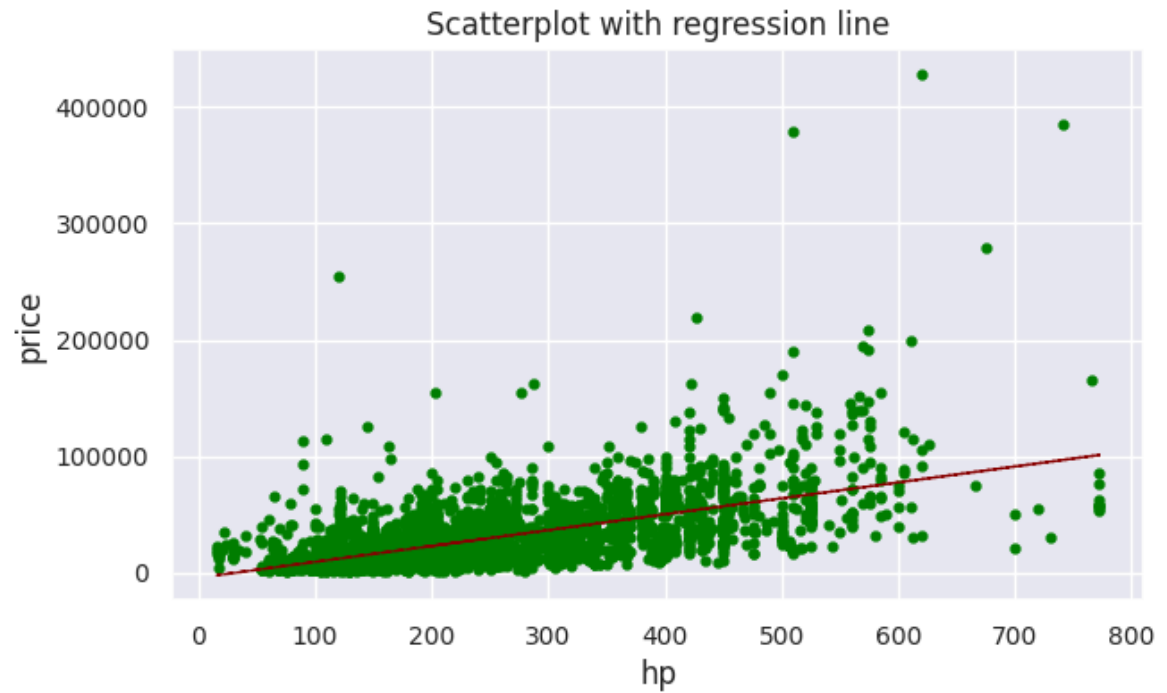
# Plot regression line
plt.plot(df_sub.hp,
         b*df_sub.hp + a,
         linewidth=1,
         linestyle='dashed',
         color='darkred')

# Add title and axes labels
plt.title('Scatterplot with regression line', fontsize=12)
plt.ylabel('price', fontsize=12)
plt.xlabel('hp', fontsize=12)

# Set fontsize of tick labels
plt.xticks(fontsize = 10)
plt.yticks(fontsize = 10)

plt.show()
```

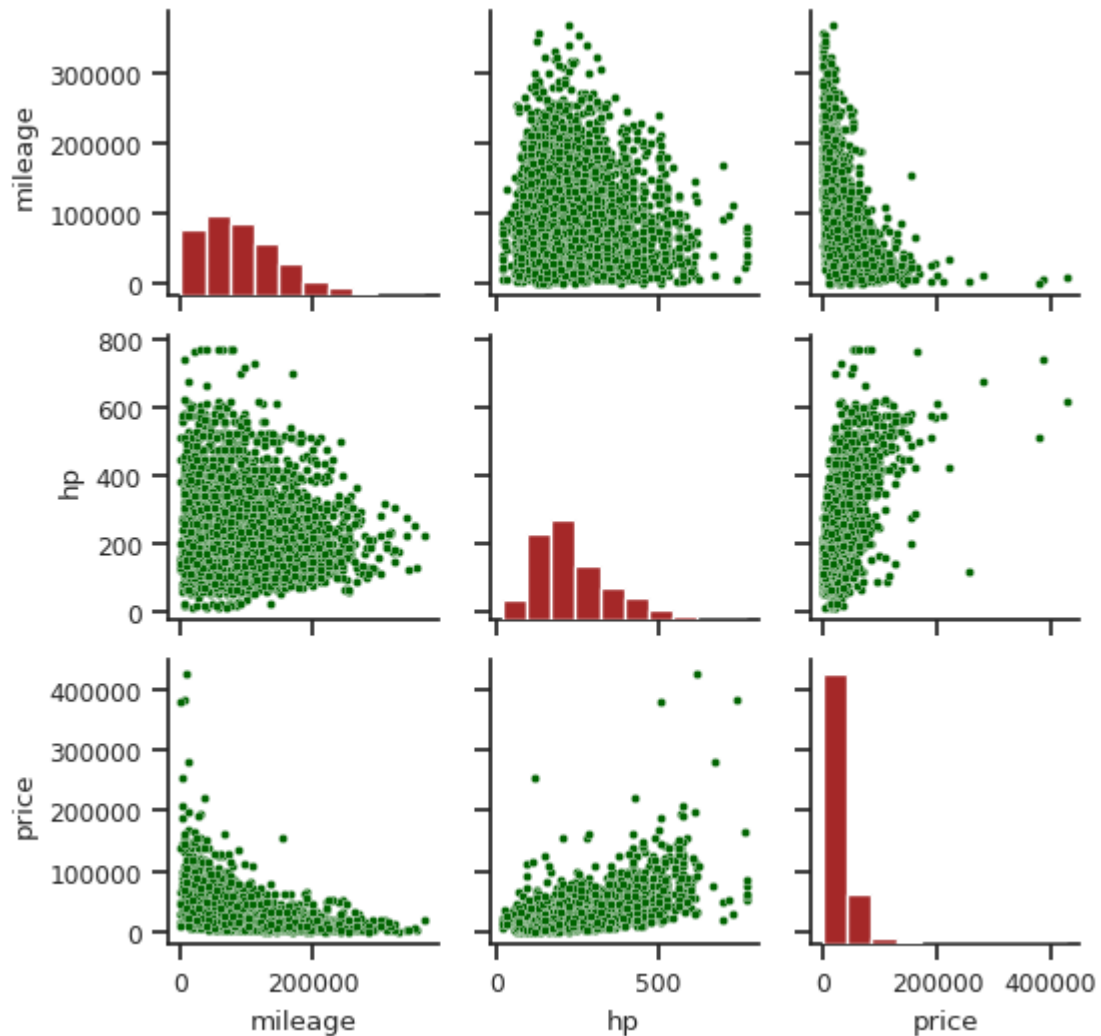
```
(3963, 15)  
136.82386275955662  
-4720.431165293197
```



Scatterplot-matrix (seaborn)

```
In [26]: sns.set_theme(style="ticks", font_scale=0.8)  
g = sns.PairGrid(df[['mileage',  
                    'hp',  
                    'price']],  
                height=1.8,  
                aspect=1)  
g.map_upper(sns.scatterplot, color='darkgreen', s=10)  
g.map_lower(sns.scatterplot, color='darkgreen', s=10)  
g.map_diag(plt.hist, color='brown')
```

```
Out[26]: <seaborn.axisgrid.PairGrid at 0x7a5dd1f3edb0>
```



Hexagonal binning plot (matplotlib)

```
In [27]: # Subset
df_sub = df.loc[(df.price <= 100000) & (df.hp <= 400)]
print(df_sub.shape)

# Plot
fig = plt.figure( figsize=(7,4) )
plt.hexbin(df_sub.hp, df_sub.price, gridsize=30, cmap='Greens')

# Set Labels
plt.xlabel('hp', fontsize=12, labelpad=10)
plt.ylabel('price', fontsize=12, labelpad=10)
plt.title('Two-dimensional histogram of price versus hp', fontsize=12, pad=10)

cb = plt.colorbar(label='count in bin')
cb.set_label('counts in bin')
```

(3555, 15)



Correlation heatmap (seaborn)

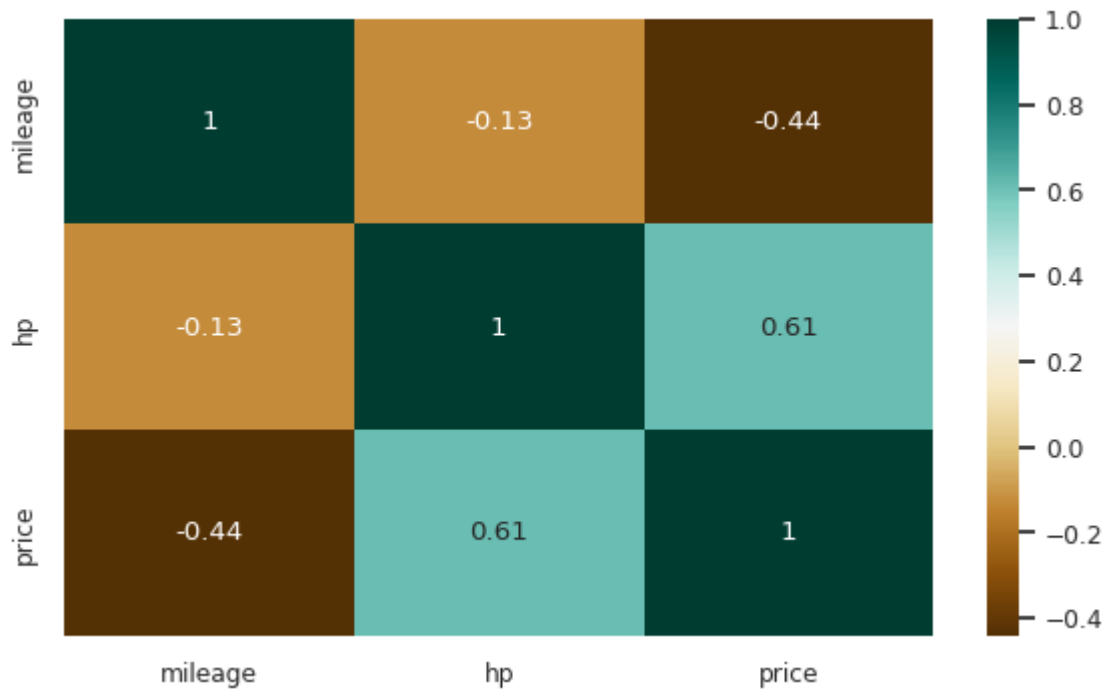
```
In [28]: # Set font size for plot
sns.set_theme(font_scale=0.8)

# Create correlation matrix as the basis for the heatmap
corr = df[['mileage',
           'hp',
           'price']].corr().round(2)
print(corr)

# Plot heatmap
plt.figure(figsize=(7,4))
sns.heatmap(corr,
            cmap="BrBG",
            annot=True)
```

	mileage	hp	price
mileage	1.00	-0.13	-0.44
hp	-0.13	1.00	0.61
price	-0.44	0.61	1.00

Out[28]: <Axes: >



Bubble plot (seaborn)

```
In [29]: # Calculate price per hp
df['price_per_hp'] = df['price'] / df['hp']

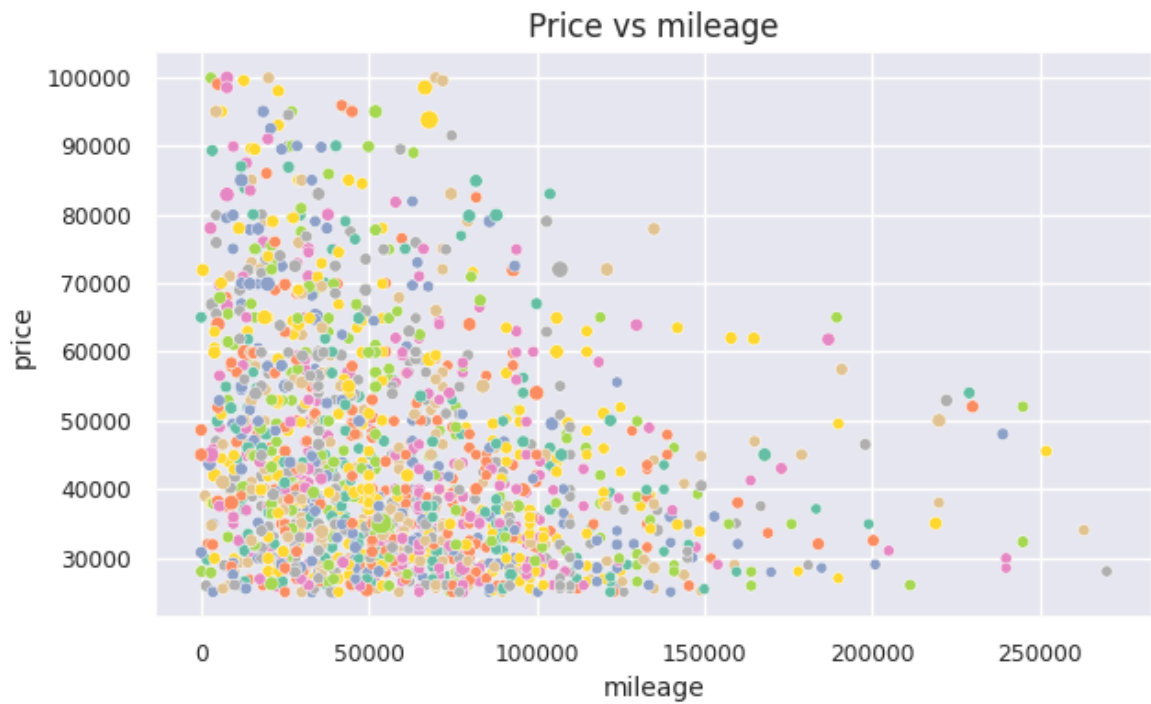
# Subset of df
df_sub = df.loc[(df['price'] >= 25000) & (df['price'] <= 100000)]

plt.figure(figsize=(7,4))
plt.ticklabel_format(style='plain')
cmap = sns.cubehelix_palette(dark=.3, light=3, as_cmap=True)

ax = sns.scatterplot(x="mileage",
                    y="price",
                    size="price_per_hp", # determines bubble size
                    hue="mileage", # determines color
                    palette="Set2",
                    data=df_sub)

# Set title and axes
ax.set_title('Price vs mileage', fontsize = 12)
ax.set_xlabel('mileage', fontsize = 10)
ax.set_ylabel('price', fontsize = 10)
ax.legend([],[], frameon=False) # skip Legend
```

Out[29]: <matplotlib.legend.Legend at 0x7a5dd0162ea0>



Jupyter notebook --footer info-- (please always provide this at the end of each notebook)

```
In [30]: import os
import platform
import socket
from platform import python_version
from datetime import datetime

print('-----')
print(os.name.upper())
print(platform.system(), '|', platform.release())
print('Datetime:', datetime.now().strftime("%Y-%m-%d %H:%M:%S"))
print('Python Version:', python_version())
print('-----')
```

```
-----
POSIX
Linux | 6.8.0-87-generic
Datetime: 2025-11-02 19:58:22
Python Version: 3.12.12
-----
```

Exploring spatial data

Libraries and settings

```
In [1]: # Libraries
import os
import folium
import geopandas as gpd
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Ignore warnings
import warnings
warnings.filterwarnings("ignore")

# Show current working directory
print(os.getcwd())
```

/home/pfrei/Dokumente/GitHub/python_exploratory_data_analysis

Read data

```
In [2]: # Polygonmap as .json-File
polys = gpd.read_file("GEN_A4_GEMEINDEN_2019_epsg4326.json")

# Show data structure and first records
print("nrows, ncols", polys.shape)
print("-----")
print("Type:", type(polys))

# Object is GeoDataFrame (includes geometry of polygons)
polys
```

nrows, ncols (162, 6)

Type: <class 'geopandas.geodataframe.GeoDataFrame'>

Out[2]:	BFS	NAME	BEZIRKSNAM	ART_TEXT	ART_CODE	geometry
0	117	Hinwil	Hinwil	Gemeinde	1	POLYGON ((8.84778 47.3241, 8.85861 47.32162, 8...
1	131	Adliswil	Horgen	Gemeinde	1	POLYGON ((8.53489 47.32502, 8.53662 47.321, 8...
2	3	Bonstetten	Affoltern	Gemeinde	1	POLYGON ((8.46026 47.33326, 8.46753 47.3341, 8...
3	154	Küsnacht (ZH)	Meilen	Gemeinde	1	POLYGON ((8.60977 47.33352, 8.61127 47.32749, ...
4	135	Kilchberg (ZH)	Horgen	Gemeinde	1	POLYGON ((8.54625 47.33441, 8.54875 47.33113, ...
...	...	...	...	...	...	...
157	192	Egg	Uster	Gemeinde	1	POLYGON ((8.70571 47.32027, 8.70343 47.31724, ...
158	115	Gossau (ZH)	Hinwil	Gemeinde	1	POLYGON ((8.77442 47.31964, 8.78551 47.31752, ...
159	29	Flurlingen	Andelfingen	Gemeinde	1	POLYGON ((8.63514 47.69119, 8.64204 47.69104, ...
160	27	Feuerthalen	Andelfingen	Gemeinde	1	POLYGON ((8.64708 47.69551, 8.65089 47.6938, 8...
161	292	Stammheim	Andelfingen	Gemeinde	1	POLYGON ((8.81793 47.65095, 8.82664 47.64894, ...

162 rows × 6 columns

Plot spatial map

```
In [3]: # Initializing the map
m = folium.Map(location=[47.44, 8.65], zoom_start=10)

# Map settings
folium.Choropleth(
    geo_data=polys,
    name='polys',
    fill_color='greenyellow'
).add_to(m)

folium.LayerControl().add_to(m)
```

```
# Plot map
m
```

Out[3]:



Plot a subset of the spatial map

```
In [4]: # Subset of map with index
idx = polys[polys['NAME'] == 'Winterthur'].index[0]
polys.iloc[[idx]]
```

Out[4]:

	BFS	NAME	BEZIRKSNAM	ART_TEXT	ART_CODE	geometry
119	230	Winterthur	Winterthur	Gemeinde	1	POLYGON ((8.76757 47.54828, 8.77696 47.5472, 8...

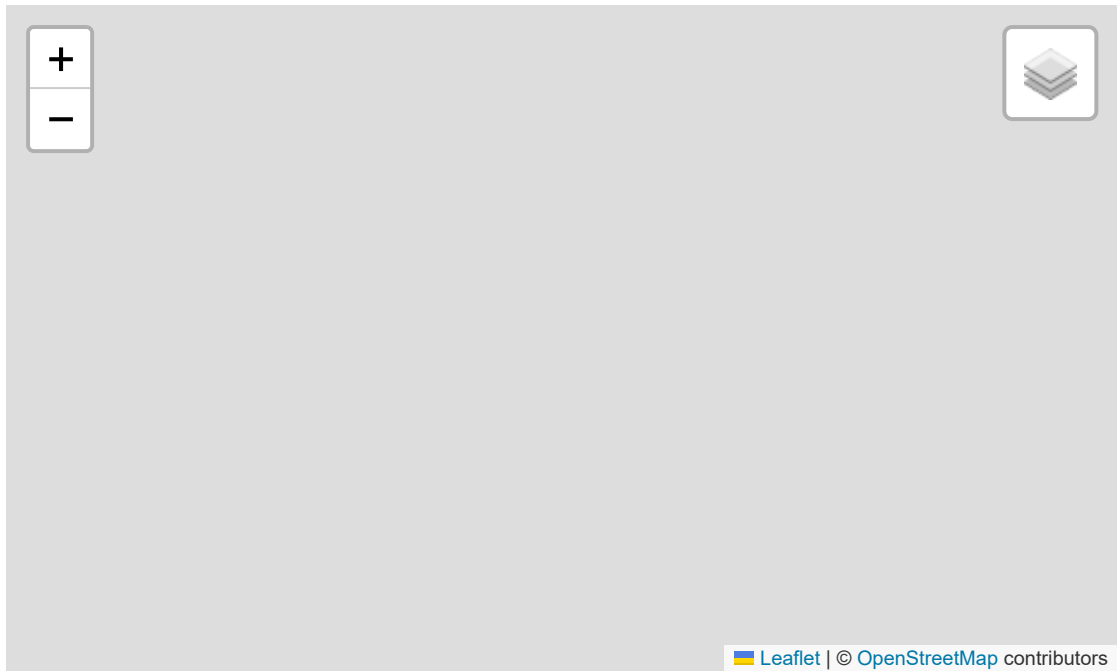
```
In [5]: # Initializing the map
m = folium.Map(location=[47.44, 8.65], zoom_start=10)

# Map settings
folium.Choropleth(
    geo_data=polys.iloc[[idx]],
    name='polys',
    fill_color='greenyellow'
).add_to(m)

folium.LayerControl().add_to(m)

# Plot map
m
```

Out[5]:



Explore attribute data

```
In [6]: # Read municipality data
data = pd.read_excel('municipalities_kt_zh_data.xlsx', index_col=None)
print(type(data))
data.head(5)
```

<class 'pandas.core.frame.DataFrame'>

Out[6]:

	BFS	municipality_name	residents	percentage_foreigners	area_km2	residents_per_km2
0	21	Adlikon	665	9.2	6.58	101.0638
1	131	Adliswil	18803	35.3	7.77	2419.9485
2	241	Aesch (ZH)	1348	15.7	5.24	257.2519
3	1	Aeugst am Albis	1941	12.7	7.91	245.3855
4	2	Affoltern am Albis	12146	27.6	10.59	1146.9310



```
In [7]: # Describe data
data.describe()
```

Out[7]:

	BFS	residents	percentage_foreigners	area_km2	residents_per_km2
count	162.000000	162.000000	162.000000	162.000000	162.000000
mean	124.969136	9269.549383	18.917284	10.251914	795.841253
std	83.345157	33263.791485	8.242690	10.075273	801.817314
min	1.000000	342.000000	5.100000	1.590000	64.950712
25%	56.250000	1892.750000	12.500000	5.060000	231.625802
50%	112.500000	4396.000000	18.300000	7.885000	552.378602
75%	195.750000	7837.750000	24.450000	12.012500	1014.056106
max	298.000000	409241.000000	46.200000	87.930000	4654.168088

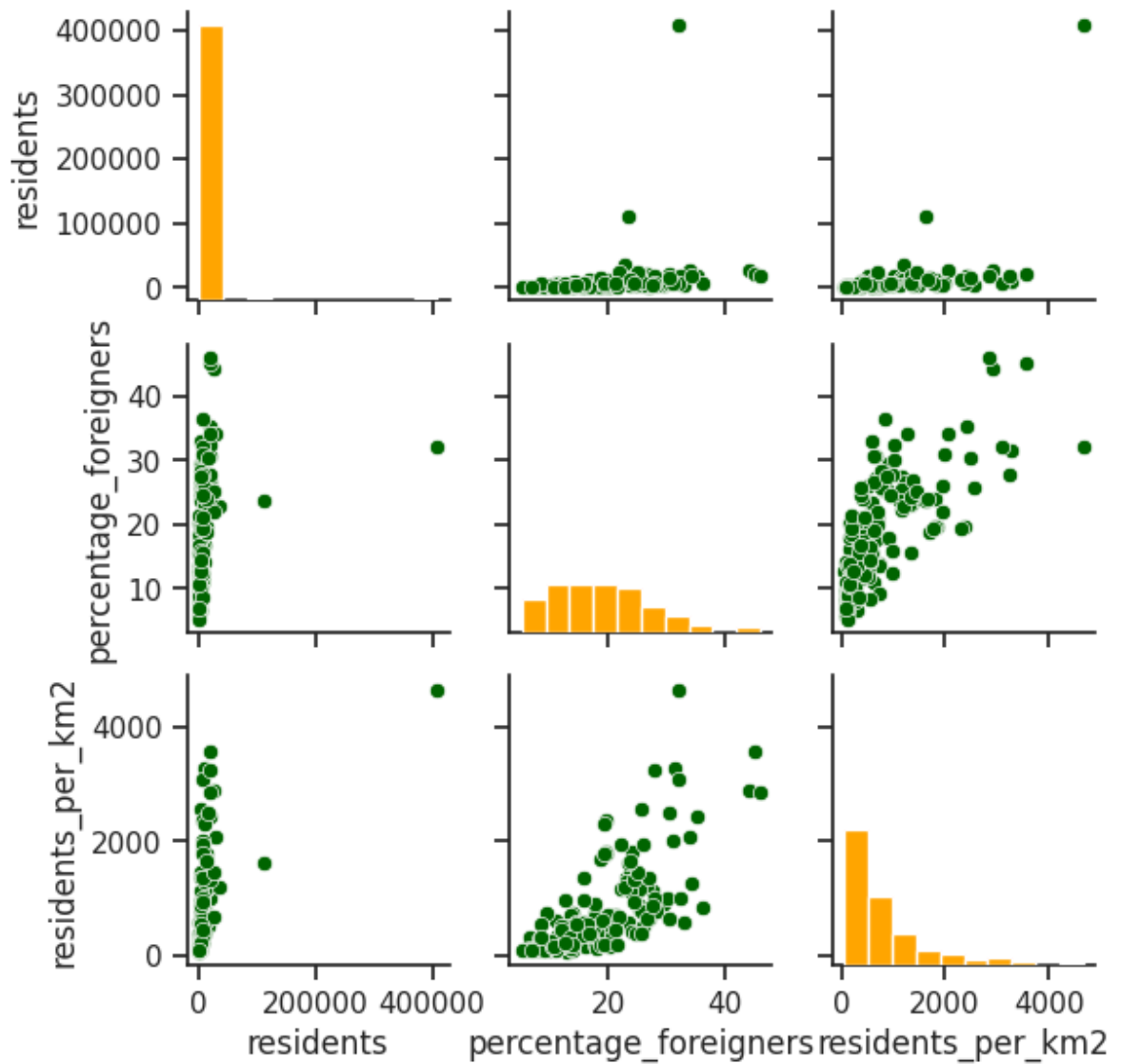


In [8]:

```
# Pairplot
sns.set(style="ticks")

g = sns.PairGrid(data[['residents', 'percentage_foreigners', 'residents_per_km2'])
g.map_upper(sns.scatterplot, color='darkgreen')
g.map_lower(sns.scatterplot, color='darkgreen')
g.map_diag(plt.hist, color='orange')

plt.show()
```



Find useful base maps

```
In [9]: # Create a map using Stamen Terrain as the basemap
m = folium.Map(location=[47.364, 8.535],
               tiles = 'CartoDBPositron')

# Alternative map designs (tiles):
# EsriWorldImagery
# EsriWorldTopoMap
# EsriWorldGrayCanvas
# CartoDBDarkMatter
# CartoDBPositron

# Display spatial map
m
```

Out[9]:



 Leaflet | © OpenStreetMap contributors © CARTO

Create choropleth map

```
In [10]: # Function for hiding original Legend
def folium_del_legend(choropleth: folium.Choropleth):
    del_list = []
    for child in choropleth._children:
        if child.startswith('color_map'):
            del_list.append(child)
            for del_item in del_list:
                choropleth._children.pop(del_item)
    return choropleth

# Geojson and data
polys = 'GEN_A4_GEMEINDEN_2019_epsg4326.json'
data = pd.read_excel('municipalities_kt_zh_data.xlsx')

# Bins for color-range of the map
bins = list(data['residents_per_km2'].quantile([0.00, 0.25, 0.50, 0.75, 1.00]))

# Initializing the map
m = folium.Map(location=[47.44, 8.65],
                zoom_start=10,
                tiles = 'EsriWorldGrayCanvas')

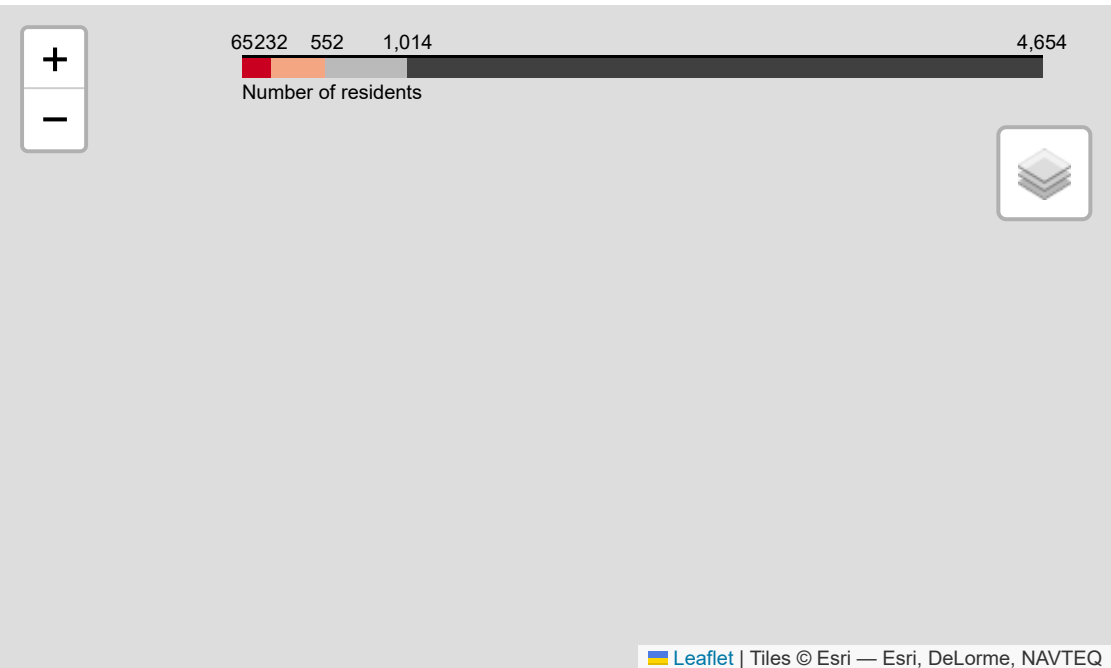
# Map-Settings
folium.Choropleth(
    geo_data=polys,
    name='choropleth',
    data=data,
    columns=['BFS', 'residents_per_km2'],
    key_on='feature.properties.BFS',
    fill_color='RdGy',
    fill_opacity=0.7,
    line_opacity=0.5,
    legend_name='Number of residents',
    bins=bins,
    reset=True
).add_to(m)
```

```
# Layer controls
folium.LayerControl(collapsed=True).add_to(m)

# Plot map
m

# Save map as HTML
# m.save('map.html')
```

Out[10]:



Jupyter notebook --footer info-- (please always provide this at the end of each notebook)

```
In [11]: import os
import platform
import socket
from platform import python_version
from datetime import datetime

print('-----')
print(os.name.upper())
print(platform.system(), '|', platform.release())
print('Datetime:', datetime.now().strftime("%Y-%m-%d %H:%M:%S"))
print('Python Version:', python_version())
print('-----')
```

```
-----
POSIX
Linux | 6.8.0-87-generic
Datetime: 2025-11-02 20:19:56
Python Version: 3.12.12
-----
```

```
In [2]: def plot_colortable(colors, title, sort_colors=True, emptycols=0):

        cell_width = 212
        cell_height = 20
        swatch_width = 40
        margin = 12
        topmargin = 40
```

```

# Sort colors by hue, saturation, value and name.
if sort_colors is True:
    by_hsv = sorted((tuple(mcolors.rgb_to_hsv(mcolors.to_rgb(color))),
                           name)
                    for name, color in colors.items())
    names = [name for hsv, name in by_hsv]
else:
    names = list(colors)

n = len(names)
ncols = 4 - emptycols
nrows = n // ncols + int(n % ncols > 0)

width = cell_width * 4 + 2 * margin
height = cell_height * nrows + margin + topmargin
dpi = 52

fig, ax = plt.subplots(figsize=(width / dpi, height / dpi), dpi=dpi)
fig.subplots_adjust(margin/width, margin/height,
                    (width-margin)/width, (height-topmargin)/height)
ax.set_xlim(0, cell_width * 4)
ax.set_ylim(cell_height * (nrows-0.5), -cell_height/2.)
ax.yaxis.set_visible(False)
ax.xaxis.set_visible(False)
ax.set_axis_off()
ax.set_title(title, fontsize=24, loc="left", pad=10)

for i, name in enumerate(names):
    row = i % nrows
    col = i // nrows
    y = row * cell_height

    swatch_start_x = cell_width * col
    text_pos_x = cell_width * col + swatch_width + 7

    ax.text(text_pos_x, y, name, fontsize=14,
            horizontalalignment='left',
            verticalalignment='center')

    ax.add_patch(
        Rectangle(xy=(swatch_start_x, y-9), width=swatch_width,
                  height=18, facecolor=colors[name], edgecolor='0.7')
    )

return fig

plot_colortable(mcolors.BASE_COLORS, "Base Colors",
                sort_colors=False, emptycols=1)
plot_colortable(mcolors.TABLEAU_COLORS, "Tableau Palette",
                sort_colors=False, emptycols=2)

plot_colortable(mcolors.CSS4_COLORS, "CSS Colors")

# Optionally plot the XKCD colors (Caution: will produce large figure)
# xkcd_fig = plot_colortable(mcolors.XKCD_COLORS, "XKCD Colors")
# xkcd_fig.savefig("XKCD_Colors.png")

plt.show()

```

Base Colors



Tableau Palette



CSS Colors

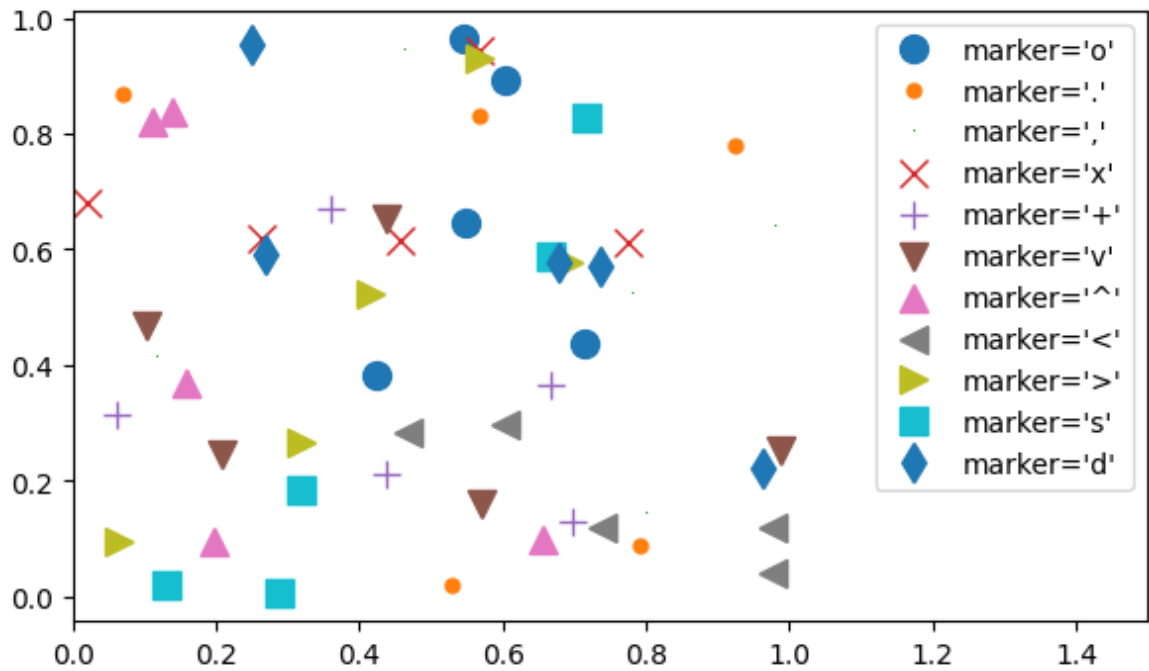


Marker symbols

```
In [3]: # Different marker symbols in one plot
rng = np.random.RandomState(0)

plt.figure(figsize=(7,4))
for marker in ['o', '.', ',', 'x', '+', 'v', '^', '<', '>', 's', 'd']:
    plt.plot(rng.rand(5), rng.rand(5), marker,
             label="marker='{0}'".format(marker), markersize=10)
plt.legend(numpoints=1, fontsize=10)
plt.xlim(0, 1.5)
```

Out[3]: (0.0, 1.5)



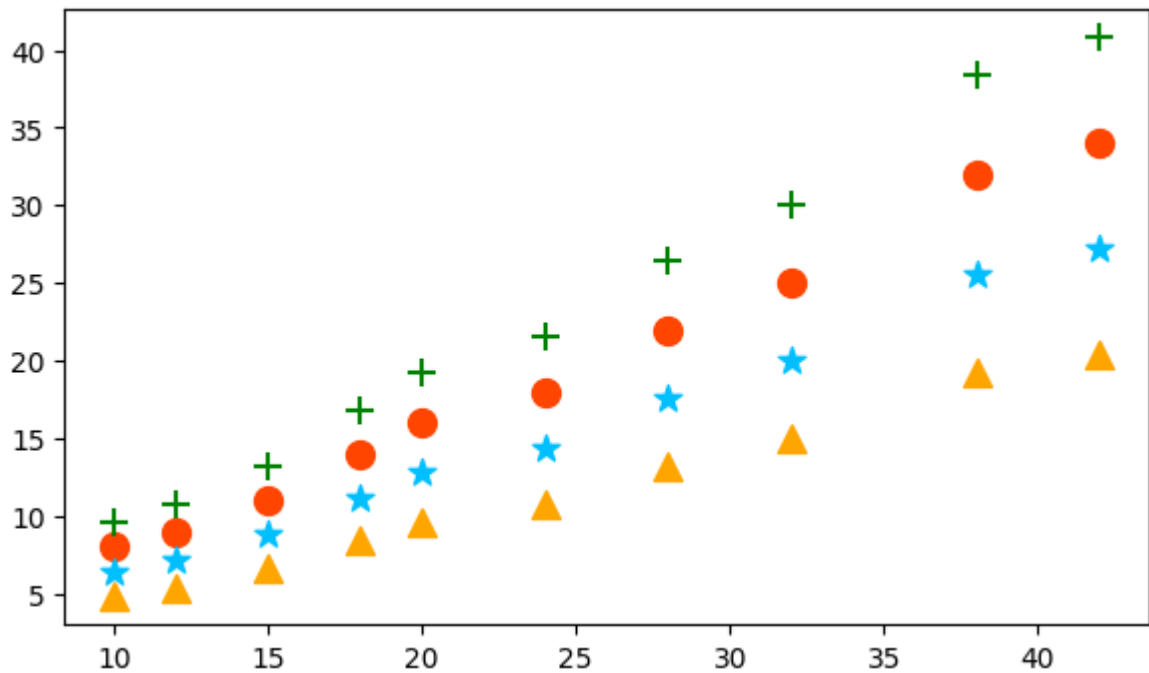
Scatterplot with individual marker symbols

```
In [4]: # Data
x = [10, 12, 15, 18, 20, 24, 28, 32, 38, 42]
y = [8, 9, 11, 14, 16, 18, 22, 25, 32, 34]
z = list(map(lambda x : x*1.2, y))
g = list(map(lambda x : x*0.8, y))
e = list(map(lambda x : x*0.6, y))

# Create a figure and axes
plt.figure(figsize=(7,4))

# Function to plot and show graph
scatter_01 = plt.scatter(x, y, s=100, marker='o', color='orangered')
scatter_02 = plt.scatter(x, z, s=100, marker='+', color='green')
scatter_03 = plt.scatter(x, g, s=100, marker='*', color='deepskyblue')
scatter_04 = plt.scatter(x, e, s=100, marker='^', color='orange')

plt.show()
```



Formatting axes and setting limits and ticks

```
In [5]: # Data
x = [10, 12, 15, 18, 20, 24, 28, 32, 38, 42]
y = [8, 9, 11, 14, 16, 18, 22, 25, 32, 34]
z = list(map(lambda x : x*1.2, y))
g = list(map(lambda x : x*0.8, y))
e = list(map(lambda x : x*0.6, y))

# Create a figure and axes
plt.figure(figsize=(7,4))

# Title and axes Labels
plt.title('Scatterplot', fontsize=12)
plt.ylabel('y-axis', fontsize=12)
plt.xlabel('x-axis', fontsize=12)

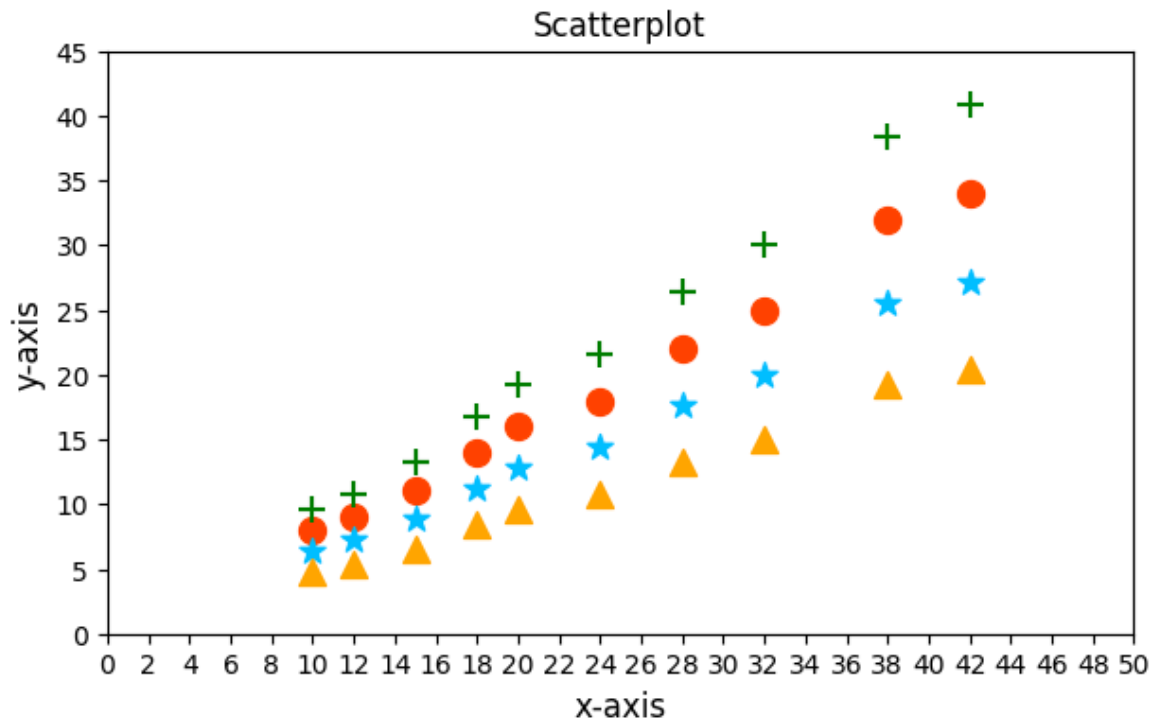
# Set tick Labels Locations
plt.xticks(np.arange(0, 60, step=2))
plt.yticks(np.arange(0, 60, step=5))

# Set fontsize of tick labels
plt.xticks(fontsize = 10)
plt.yticks(fontsize = 10)

# Set x, y-axis limits
plt.xlim(0, 50)
plt.ylim(0, 45)

# Function to plot and show graph
scatter_01 = plt.scatter(x, y, s=100, marker='o', color='orangered')
scatter_02 = plt.scatter(x, z, s=100, marker='+', color='green')
scatter_03 = plt.scatter(x, g, s=100, marker='*', color='deepskyblue')
scatter_04 = plt.scatter(x, e, s=100, marker='^', color='orange')

plt.show()
```



Adding a legend

```
In [6]: # Data
x = [10, 12, 15, 18, 20, 24, 28, 32, 38, 42]
y = [8, 9, 11, 14, 16, 18, 22, 25, 32, 34]
z = list(map(lambda x : x*1.2, y))
g = list(map(lambda x : x*0.8, y))
e = list(map(lambda x : x*0.6, y))

# Create a figure and axes
plt.figure(figsize=(7,4))

# Title and axes Labels
plt.title('Scatterplot', fontsize=12)
plt.ylabel('y-axis', fontsize=12)
plt.xlabel('x-axis', fontsize=12)

# Set tick Labels Locations
plt.xticks(np.arange(0, 60, step=2))
plt.yticks(np.arange(0, 60, step=5))

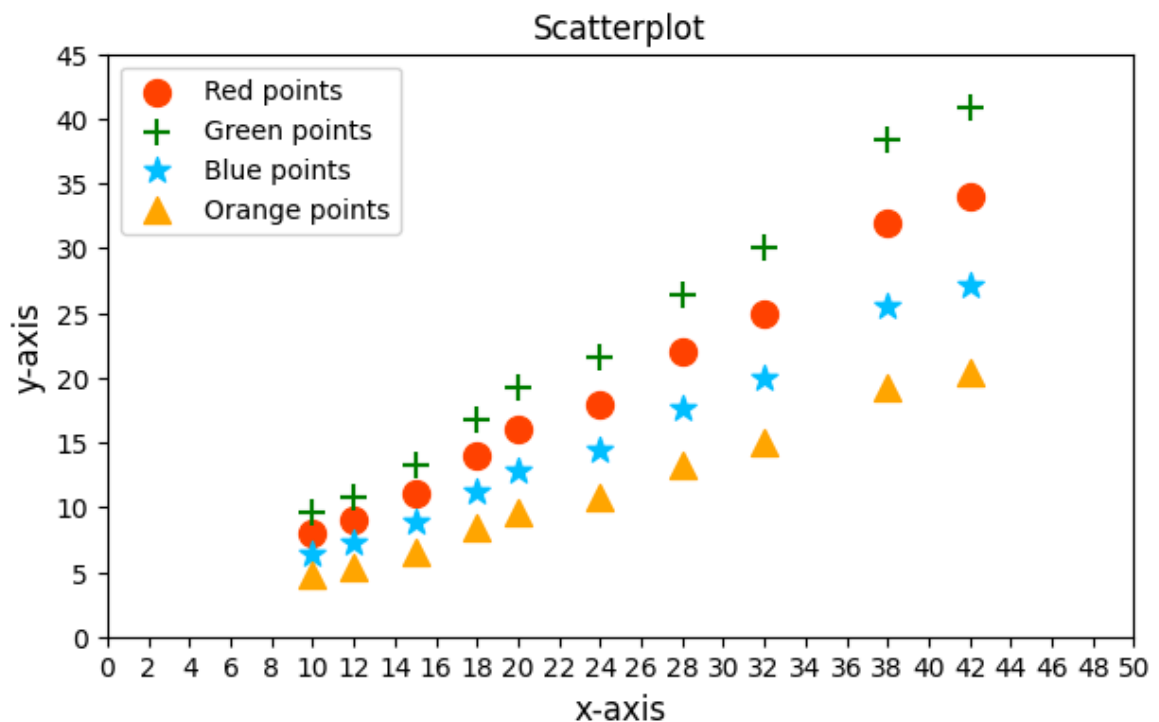
# Set fontsize of tick labels
plt.xticks(fontsize = 10)
plt.yticks(fontsize = 10)

# Set x, y-axis limits
plt.xlim(0, 50)
plt.ylim(0, 45)

# Function to plot and show graph
scatter_01 = plt.scatter(x, y, s=100, marker='o', color='orangered')
scatter_02 = plt.scatter(x, z, s=100, marker='+', color='green')
scatter_03 = plt.scatter(x, g, s=100, marker='*', color='deepskyblue')
scatter_04 = plt.scatter(x, e, s=100, marker='^', color='orange')
```

```
# Add Legend
plt.legend(handles = [scatter_01, scatter_02, scatter_03, scatter_04],
          labels = ['Red points', 'Green points', 'Blue points', 'Orange points'],
          loc="upper left",
          fontsize=10)

plt.show()
```



Adding a text annotation

```
In [7]: # Data
x = [10, 12, 15, 18, 20, 24, 28, 32, 38, 42]
y = [8, 9, 11, 14, 16, 18, 22, 25, 32, 34]
z = list(map(lambda x : x*1.2, y))
g = list(map(lambda x : x*0.8, y))
e = list(map(lambda x : x*0.6, y))

# Create a figure and axes
plt.figure(figsize=(7,4))

# Title and axes Labels
plt.title('Scatterplot', fontsize=12)
plt.ylabel('y-axis', fontsize=12)
plt.xlabel('x-axis', fontsize=12)

# Set tick Labels Locations
plt.xticks(np.arange(0, 60, step=2))
plt.yticks(np.arange(0, 60, step=5))

# Set fontsize of tick labels
plt.xticks(fontsize = 10)
plt.yticks(fontsize = 10)

# Set x, y-axis limits
plt.xlim(0, 50)
plt.ylim(0, 45)
```

```

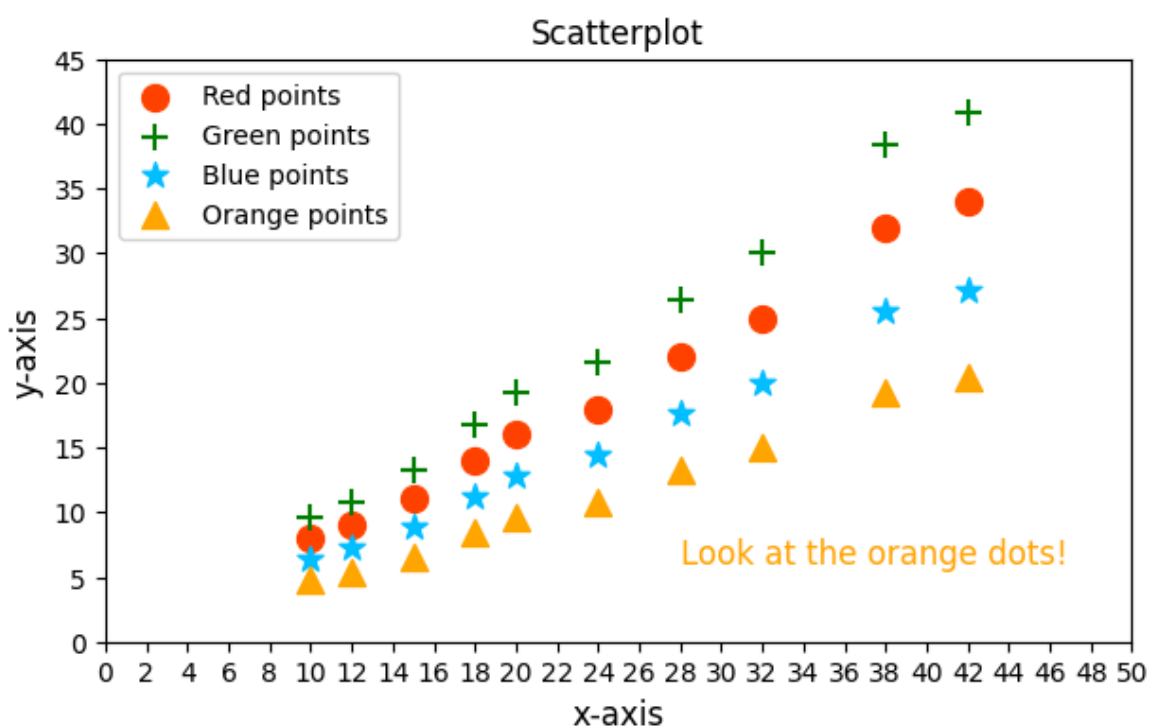
# Function to plot and show graph
scatter_01 = plt.scatter(x, y, s=100, marker='o', color='orangered')
scatter_02 = plt.scatter(x, z, s=100, marker='+', color='green')
scatter_03 = plt.scatter(x, g, s=100, marker='*', color='deepskyblue')
scatter_04 = plt.scatter(x, e, s=100, marker='^', color='orange')

# Add Legend
plt.legend(handles = [scatter_01, scatter_02, scatter_03, scatter_04],
          labels = ['Red points', 'Green points', 'Blue points', 'Orange points'],
          loc="upper left",
          fontsize=10)

# Add text annotation
plt.text(28, 6, s="Look at the orange dots!", fontsize=12, color="orange")

plt.show()

```



Adding a grid

```

In [8]: # Data
x = [10, 12, 15, 18, 20, 24, 28, 32, 38, 42]
y = [8, 9, 11, 14, 16, 18, 22, 25, 32, 34]
z = list(map(lambda x : x*1.2, y))
g = list(map(lambda x : x*0.8, y))
e = list(map(lambda x : x*0.6, y))

# Create a figure and axes
plt.figure(figsize=(7,4))

# Title and axes Labels
plt.title('Scatterplot', fontsize=12)
plt.ylabel('y-axis', fontsize=12)
plt.xlabel('x-axis', fontsize=12)

# Set tick Labels Locations

```

```

plt.xticks(np.arange(0, 60, step=2))
plt.yticks(np.arange(0, 60, step=5))

# Set fontsize of tick labels
plt.xticks(fontsize = 10)
plt.yticks(fontsize = 10)

# Set x, y-axis limits
plt.xlim(0, 50)
plt.ylim(0, 45)

# Function to plot and show graph
scatter_01 = plt.scatter(x, y, s=100, marker='o', color='orangered')
scatter_02 = plt.scatter(x, z, s=100, marker='+', color='green')
scatter_03 = plt.scatter(x, g, s=100, marker='*', color='deepskyblue')
scatter_04 = plt.scatter(x, e, s=100, marker='^', color='orange')

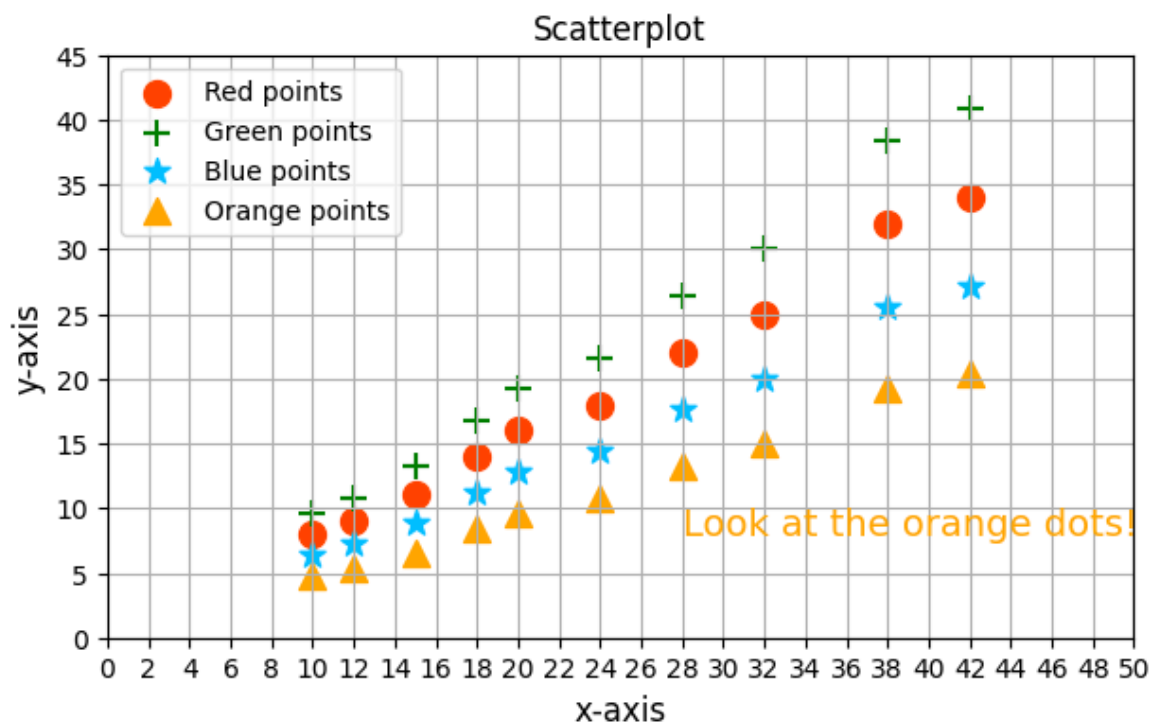
# Add Legend
plt.legend(handles = [scatter_01, scatter_02, scatter_03, scatter_04],
          labels = ['Red points', 'Green points', 'Blue points', 'Orange points'],
          loc="upper left",
          fontsize=10)

# Add text annotation
plt.text(28, 8, s="Look at the orange dots!", fontsize=14, color="orange")

# Adding a grid to the plot
plt.grid()

plt.show()

```



Creating multiplots with .subplots()

```

In [9]: # Define 2x2 subplots and whether x- and y-axes should be shared
fig, ax = plt.subplots(2, 2, figsize=(7,4), sharex=True, sharey=True)

```

```

ax[0, 0].scatter(x, y, s=100, marker='o', color='orangered') # row=0, col=0
ax[1, 0].scatter(x, z, s=100, marker='+', color='green') # row=1, col=0
ax[0, 1].scatter(x, g, s=100, marker='*', color='deepskyblue') # row=0, col=1
ax[1, 1].scatter(x, e, s=100, marker='^', color='orange') # row=1, col=1

# Adding title
ax[0, 0].set_title('Figure at [0, 0]', fontsize = 12)
ax[1, 0].set_title('Figure at [1, 0]', fontsize = 12)
ax[0, 1].set_title('Figure at [0, 1]', fontsize = 12)
ax[1, 1].set_title('Figure at [1, 1]', fontsize = 12)

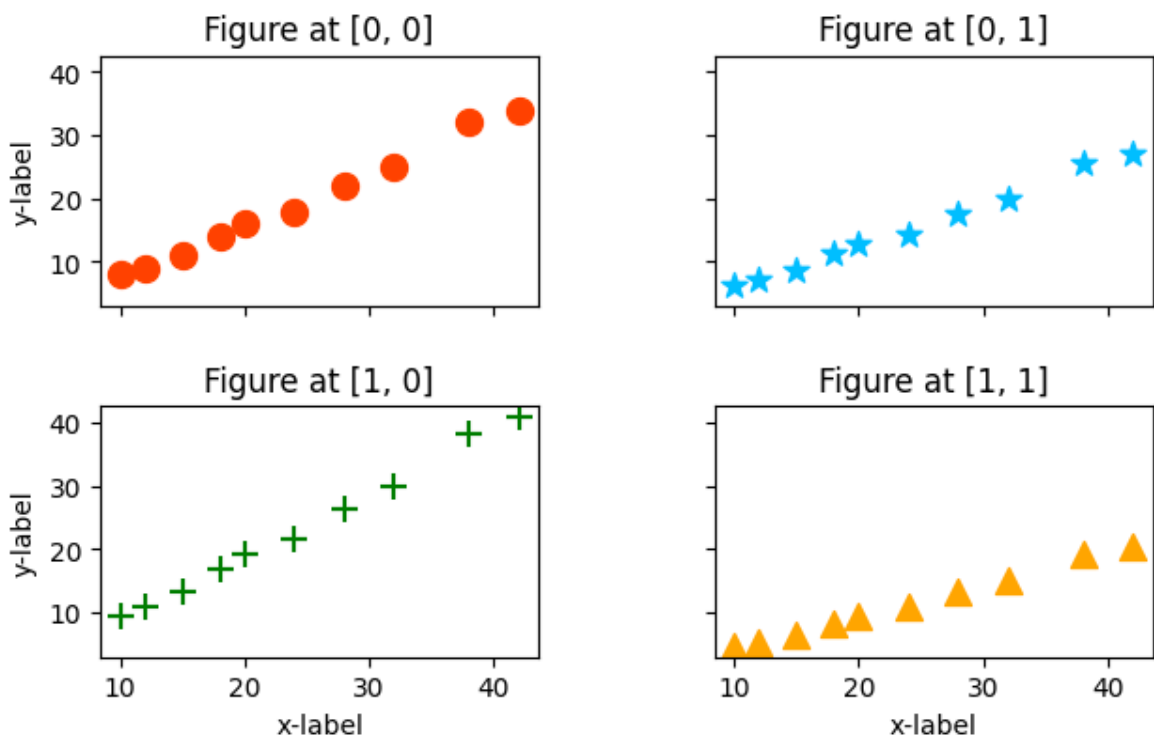
# Adding x-labels
ax[0, 0].set_xlabel('', fontsize = 10)
ax[1, 0].set_xlabel('x-label', fontsize = 10)
ax[0, 1].set_xlabel('', fontsize = 10)
ax[1, 1].set_xlabel('x-label', fontsize = 10)

# Adding y-labels
ax[0, 0].set_ylabel('y-label', fontsize = 10)
ax[1, 0].set_ylabel('y-label', fontsize = 10)
ax[0, 1].set_ylabel('', fontsize = 10)
ax[1, 1].set_ylabel('', fontsize = 10)

# Set the spacing between subplots
plt.subplots_adjust(left=0.1,
                    bottom=0.1,
                    right=0.9,
                    top=0.9,
                    wspace=0.4,
                    hspace=0.4)

plt.show()

```



Saving figure to file

```
In [10]: # Define 2x2 subplots and whether x- and y-axes should be shared
fig, ax = plt.subplots(2, 2, figsize=(7,4), sharex=True, sharey=True)

ax[0, 0].scatter(x, y, s=100, marker='o', color='orangered') # row=0, col=0
ax[1, 0].scatter(x, z, s=100, marker='+', color='green') # row=1, col=0
ax[0, 1].scatter(x, g, s=100, marker='*', color='deepskyblue') # row=0, col=1
ax[1, 1].scatter(x, e, s=100, marker='^', color='orange') # row=1, col=1

# Adding title
ax[0, 0].set_title('Figure at [0, 0]', fontsize = 12)
ax[1, 0].set_title('Figure at [1, 0]', fontsize = 12)
ax[0, 1].set_title('Figure at [0, 1]', fontsize = 12)
ax[1, 1].set_title('Figure at [1, 1]', fontsize = 12)

# Adding x-labels
ax[0, 0].set_xlabel('', fontsize = 10)
ax[1, 0].set_xlabel('x-label', fontsize = 10)
ax[0, 1].set_xlabel('', fontsize = 10)
ax[1, 1].set_xlabel('x-label', fontsize = 10)

# Adding y-labels
ax[0, 0].set_ylabel('y-label', fontsize = 10)
ax[1, 0].set_ylabel('y-label', fontsize = 10)
ax[0, 1].set_ylabel('', fontsize = 10)
ax[1, 1].set_ylabel('', fontsize = 10)

# Set the spacing between subplots
plt.subplots_adjust(left=0.1,
                    bottom=0.1,
                    right=0.9,
                    top=0.9,
                    wspace=0.4,
                    hspace=0.4)

# Save figure above to file
plt.savefig('myplot.png')

# Close connections
plt.close()
```

Jupyter notebook --footer info-- (please always provide this at the end of each notebook)

```
In [11]: import os
import platform
import socket
from platform import python_version
from datetime import datetime

print('-----')
print(os.name.upper())
print(platform.system(), '|', platform.release())
print('Datetime:', datetime.now().strftime("%Y-%m-%d %H:%M:%S"))
print('Python Version:', python_version())
print('-----')
```

POSIX

Linux | 6.8.0-87-generic

Datetime: 2025-11-02 20:20:36

Python Version: 3.12.12
